



Escola Supercomputador Santos Dumont - 2026

Programação com MPI

Escola de Computação Santos Dumont - 2026

Ementa

- Introdução
- Plataformas de Hardware
- Modelos de Programação
- Comunicação Assíncrona e Síncrona
- Avaliação de desempenho - Conceitos básicos
- Apresentação do MPI

Bibliografia



<https://www.casadocodigo.com.br/products/livro-programacao-paralela>

Escola de Computação Santos Dumont - 2026

Google Colab

- Os exemplos, notebooks e slides do minicurso estão disponíveis no seguinte endereço:
- <https://github.com/Programacao-Paralela-e-Distribuida/MPI/>



Escola Supercomputador Santos Dumont - 2026

Arquiteturas Paralelas

Arquiteturas Paralelas

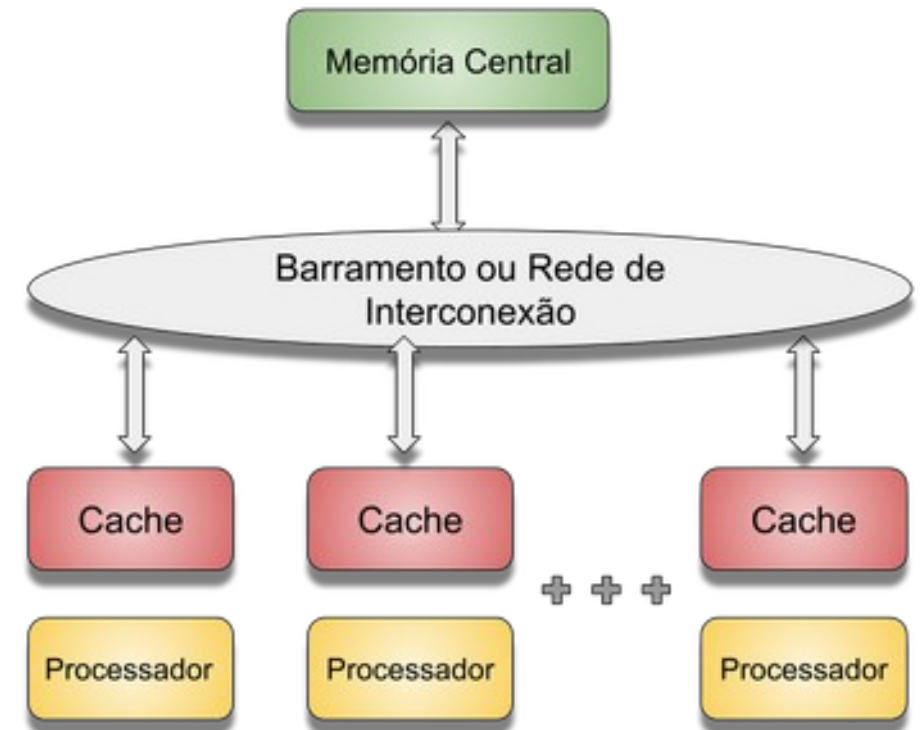
- As arquiteturas paralelas são aquelas capazes de explorar o paralelismo existente nas aplicações e podem ser organizadas de diversas maneiras.
- As arquiteturas paralelas podem explorar o paralelismo em diversos níveis, algumas vezes de forma transparente ao usuário, outras vezes não.
- Para cada tipo de arquitetura existe um paradigma de programação mais adequado, de modo a obter a maior aceleração possível, quando comparado com a execução sequencial da aplicação.

Classificação de Flynn

- **SISD (Single Instruction Single Data Stream)**
 - São os processadores convencionais, onde apenas um processo ou *thread* é executado por vez, podendo fazer uso de técnicas de exploração de paralelismo temporal (*pipeline, superpipelined*) ou espacial (superescalares) no nível de instrução, de forma transparente ao usuário.
- **SIMD (Single Instruction Multiple Data Streams)**
 - São os processadores vetoriais (supercomputadores Cray, NEC), unidades funcionais vetoriais (Intel AVX-512), arquiteturas SIMD (processamento de imagem), *tensor cores* (Google TPUs).
- **MIMD (Multiple Instruction Multiple Data Streams)**
 - Podem ser de memória compartilhada, como os multiprocessadores (*multicores*), o de memória distribuída, como os multicomputadores (*clusters*).

Arquiteturas de Memória Compartilhada

- Cada processador tem acesso a um espaço de endereçamento global comum a todos.
- Processos ou *threads* trocam informações diretamente na memória, sem necessidade de comunicação explícita com troca de mensagens.
- Os processadores fazem uso da memória cache para diminuir a contenção no acesso à memória.



Arquiteturas de Memória Compartilhada

- As suas principais vantagens são:
 - O código de programas sequenciais podem ser facilmente adaptados para versões paralelas.
 - A comunicação entre processos ou *threads* é bastante eficiente.
 - Acesso rápido à memória comparado a arquiteturas distribuídas.
- As suas principais desvantagens são:
 - Uso de primitivas de sincronização para assegurar um resultado correto para a computação.
 - Necessidade de mecanismos de coerência de cache para manter os dados atualizados entre caches locais.
 - O aumento de processadores pode causar contenção de memória e gargalos no barramento.

Arquiteturas de Memória Compartilhada

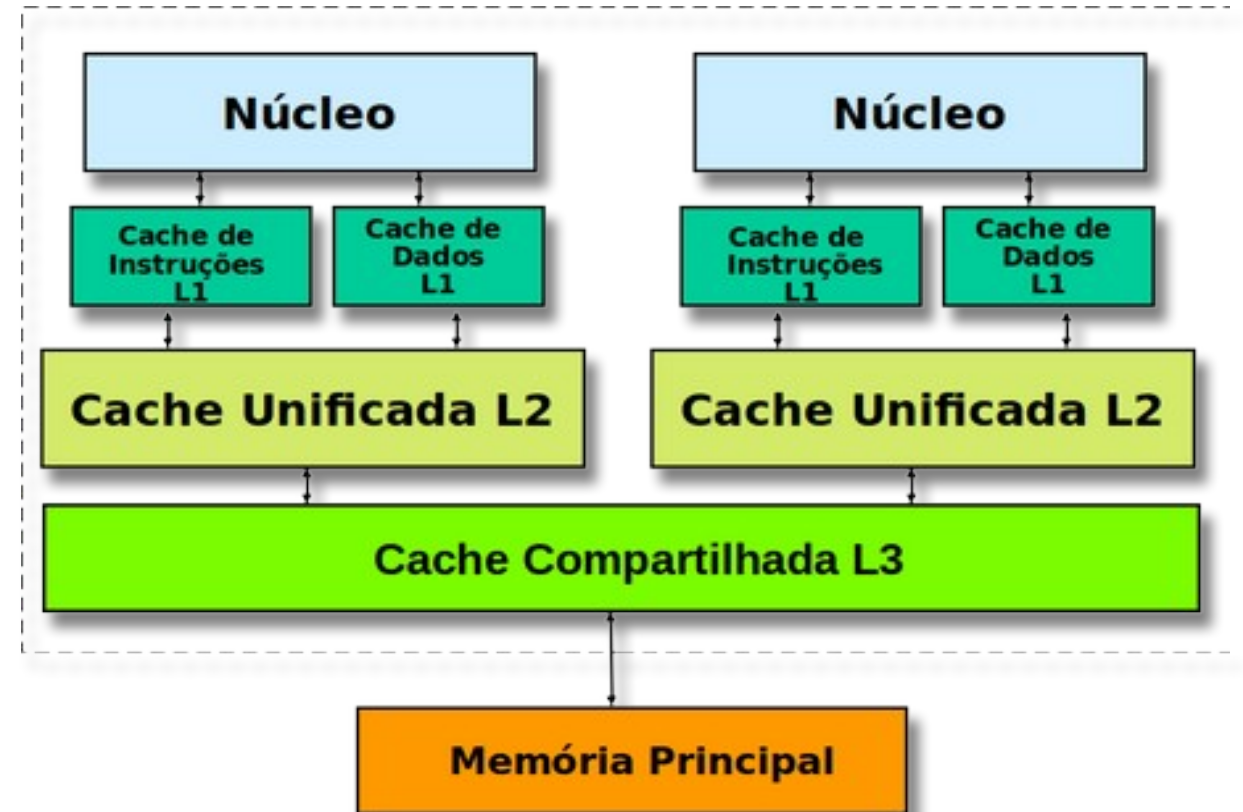
- Essas arquiteturas possuem diversas classificações e variações em sua construção, tais como:
 - UMA (Uniform Memory Access): São arquiteturas com memória única global, onde o tempo de acesso à memória é uniforme para todos os nós de processamento.
 - NUMA (Non-uniform Memory Access): A memória é dividida em blocos locais a cada processador do sistema, ligados por uma rede de interconexão. O acesso aos dados na memória local é muito mais rápido do que o acesso aos dados em blocos de memória remotos.

Arquiteturas Multicore

- Conhecidas como *chip multiprocessing*, são caracterizadas pela existência de diversos cores (*núcleos*) processadores em um mesmo encapsulamento, compartilhando uma memória cache e a memória principal.
- Esse tipo de arquitetura pode ser expandido para vários chips, cada um com vários núcleos, compartilhando uma mesma memória global, no que é chamado *multiprocessamento simétrico* (SMP).
- A comunicação entre as *threads* é feita através de variáveis na memória compartilhada (ou na memória cache no caso dos multicore), necessitando de intervenção explícita do programador para a coordenação do paralelismo.

Arquiteturas Multicore

- Vários níveis de cache existem dentro da mesma pastilha, com os níveis mais altos (L1 e L2) exclusivos de cada núcleo e o nível mais inferior compartilhado por todos os núcleos.
- Os processadores mais modernos possuem caches de até 36 MB e velocidade de relógio com até 5,0 GHz.

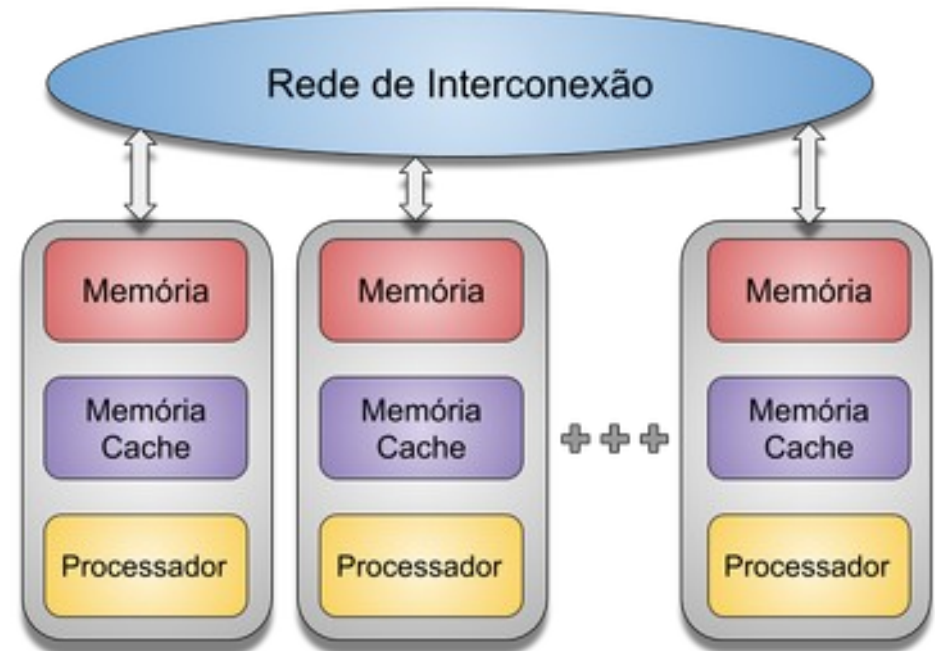


Linguagens e Frameworks

- **Linguagem 'C/C++' + pThreads:** Uma biblioteca padrão para criar e gerenciar threads em sistemas POSIX.
- **Linguagem 'C/C++' ou Fortran + OpenMP:** Uma API que adiciona diretivas ao código C/C++ para especificar paralelismo, gerenciamento de threads e sincronização.
- **Java:** Possui suporte nativo para multithreading através da biblioteca `java.lang.Thread` e frameworks como `java.util.concurrent`.
- **Python:** Embora seja uma linguagem interpretada, oferece bibliotecas como `threading` e `multiprocessing` para programação paralela. No entanto, o GIL (Global Interpreter Lock) limita o paralelismo verdadeiro em código puramente computacional. Existem bibliotecas com `NumPy` e `SciPy` para programação científica.

Arquiteturas de Memória Distribuída

- Cada processador enxerga apenas o seu espaço de memória.
- Apresenta como vantagens serem altamente escaláveis e permitirem a construção de processadores maciçamente paralelos.
- Nesse tipo de arquitetura a comunicação entre os processadores é feita através de troca de mensagens.
- A troca de mensagens resolve tanto o problema da comunicação dos processadores como o da sincronização.



Arquiteturas de Memória Distribuída

- As suas grandes desvantagens são a necessidade de se fazer uma boa distribuição de carga entre os processadores, quer seja automaticamente, quer seja manualmente.
- É necessário evitar as situações de *deadlock*, tanto no nível de aplicação como no nível de sistema operacional, possíveis de ocorrer quando há espera circular pelo envio e recebimento de mensagens.
- Além disso, é um modelo de programação menos natural, demandando diversas modificações em relação ao código convencional sequencial.

Programação Paralela

- MPI (Message Passing Interface):
 - API padrão de fato para programação em clusters e supercomputadores.
 - Oferece um conjunto rico de funções para comunicação ponto-a-ponto e coletiva, além de mecanismos de sincronização.
 - Suporta uma ampla variedade de linguagens, incluindo C, C++, Fortran e outras.
- PVM (Parallel Virtual Machine):
 - Uma das primeiras APIs para programação distribuída, ainda utilizada em alguns ambientes.
 - Oferece funcionalidades similares ao MPI, mas com uma sintaxe diferente.
 - Utilizada hoje em dia apenas em contextos educacionais.

Lista de Supercomputadores

- A lista Top500 (<https://www.top500.org>) relaciona os computadores mais rápidos do mundo.
- Essa lista contém detalhes sobre o desempenho, consumo e características dos computadores mais rápidos da atualidade.
- Detalhes como o tipo de processador, uso de aceleradores, quantidade de memória e rede de interconexão são apresentados.
- Os computadores listados apresentam uma mistura dos conceitos de memória compartilhada e distribuída.



Escola Supercomputador Santos Dumont - 2026

Programação Paralela

Programação Paralela

- A programação paralela cuida da execução coordenada de tarefas para a solução de um problema.
- Apenas existência de vários processos ou *threads* executando simultaneamente não caracteriza a programação paralela.
- Para isso é necessário que os processos ou *threads* realizem atividades coordenadas, comunicando-se e trocando as informações necessárias.
- Essa comunicação pode ser feita através de uma memória compartilhada, acessível a todos os processos e *threads* ou, quando isso não for possível, através de troca de mensagens enviadas por uma rede de comunicação.

Programação por Memória Compartilhada

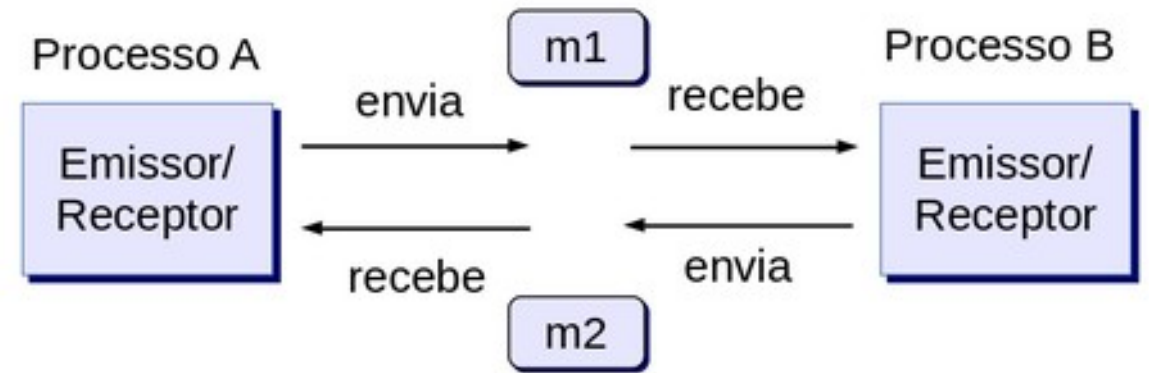
- Neste paradigma, a comunicação entre as tarefas é realizada por meio de variáveis armazenadas em um espaço de memória compartilhado. As tarefas da aplicação podem ser mapeadas em **processos ou threads**.
- No caso de **processos**, é necessário requisitar ao sistema operacional a definição de um espaço de memória compartilhada por mais de um processo. Este compartilhamento entre processos pode ser complexo e pouco eficiente.
- No caso do uso de **threads**, já existe um espaço de memória compartilhado entre as **threads** no mesmo processo que as criou. Esse acesso é rápido e de baixa complexidade. Por isso, o uso de **threads** é frequente no paradigma de memória compartilhada.

Programação por Memória Compartilhada

- Quando duas ou mais *threads* de um programa precisam trocar informações entre si, o programador define variáveis de acesso compartilhado entre elas.
- Desse modo, quando uma *thread* tem um valor novo para ser enviado para as outras *threads*, esse valor é simplesmente escrito em uma variável compartilhada, independentemente do fato de as demais *threads* estarem esperando por esse valor naquele momento.
- Isso pode gerar um problema de **condição de corrida**, exigindo o uso de primitivas de **sincronização** para resolvê-lo.

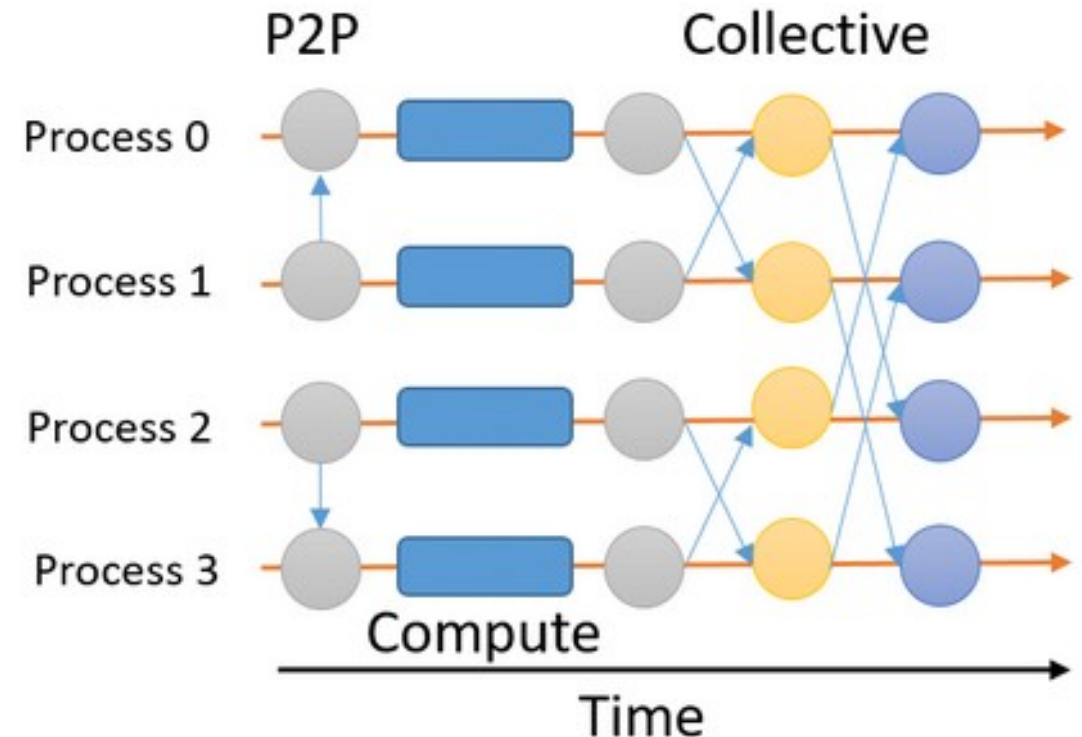
Programação por Troca de Mensagens

- Neste paradigma, a comunicação entre as tarefas da aplicação é feita por meio de rotinas de envio e recebimento de mensagens.
- As tarefas são normalmente mapeadas em processos, cada um com sua memória privada.



Programação por Troca de Mensagens

- A figura ilustra diversos processos em execução, trocando informações para a solução de um problema.
- A comunicação pode ser ponto-a-ponto, envolvendo dois processos, ou coletiva, quando mais de dois processos trocam mensagens.



Programação por Troca de Mensagens

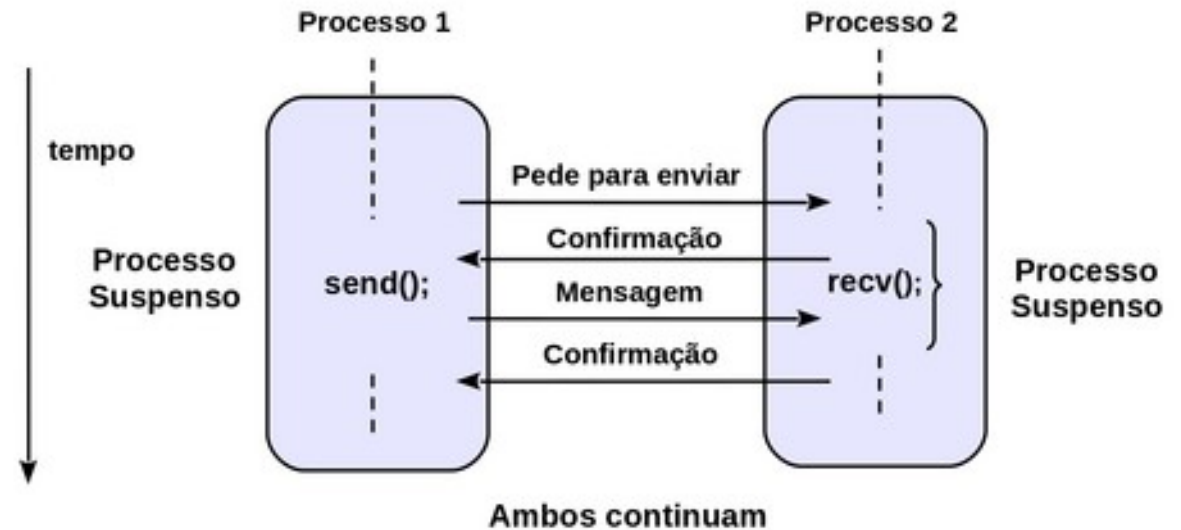
- Como não há um espaço de memória compartilhada, o problema de condição de corrida não ocorre, embora também possa haver **deadlock**.
- O **deadlock** acontece quando há um ou mais processos esperando por uma mensagem de um processo, que por sua vez está esperando por uma mensagem a ser enviada pelos demais processos.
- Há dois modos principais de comunicação no paradigma de programação por troca de mensagens: **síncrono** e **assíncrono**.

Comunicação Síncrona

- No modo de comunicação síncrona tanto o envio como a recepção de mensagens são bloqueantes.
- Ou seja, a rotina de envio não retorna para o programa principal enquanto a mensagem não for completamente enviada.
- E a função de recepção também não retorna enquanto a mensagem não estiver disponível para o usuário no espaço de memória reservado na aplicação para isso.
- Ao se concluir a operação de **envio**, as estruturas de dados utilizadas no programa do usuário para o envio da mensagem podem ser reutilizadas com segurança.
- Ao término da operação de **recepção** temos a certeza que os dados recebidos estão disponíveis para a aplicação.

Comunicação Síncrona

- Esse modelo de comunicação requer, portanto, um protocolo de quatro fases para a sua implementação:
 - pedido de autorização para transmitir;
 - recebimento da autorização;
 - transmissão da mensagem;
 - recebimento da mensagem de reconhecimento.



Comunicação Síncrona

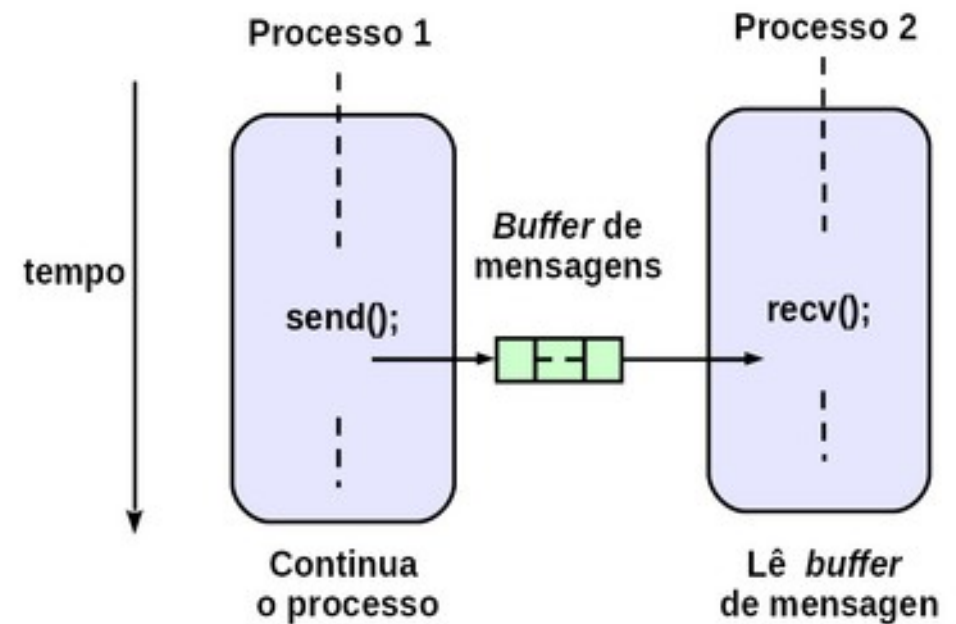
- Esse modo de comunicação é simples e seguro.
- Também permite a sincronização entre os processos, ou seja, quando um processo acaba de transmitir uma mensagem, sabe que o processo de destino acabou de receber essa mensagem.
- Esse modo, contudo, elimina a possibilidade de haver superposição entre a computação e a transmissão das mensagens, diminuindo o desempenho da aplicação paralela.

Comunicação Assíncrona

- A principal característica da comunicação assíncrona é a desvinculação do instante de tempo no qual ocorrem as operações de envio e recepção da mensagem.
- Isso pode ser feito com o uso de áreas (*buffers*) destinadas ao armazenamento da mensagem enviada.
- A computação prossegue, enquanto a transmissão dos dados é realizada através da rede de comunicação.
- A operação de envio (*send*) se completa a partir da cópia de todo o conteúdo da mensagem do espaço de endereçamento do usuário para uma área correspondente (*buffer*) da biblioteca ou do sistema operacional.

Comunicação Assíncrona

- Isso libera as estruturas de dados do programa do usuário para serem utilizadas por novas operações de envio de mensagens.
- Há outras formas de implementação da comunicação assíncrona, como por exemplo o uso de rotinas de envio e/ou recepção não-bloqueantes.



Comunicação Assíncrona

- Quando a operação de envio é **não-bloqueante**, ela retorna imediatamente, dando apenas início ao processo de transmissão da mensagem.
- Isso nada garante e não autoriza a reutilização das estruturas de dados do programa para o envio de uma nova mensagem.
- Quando a operação de recepção é **não-bloqueante**, ela apenas indica a intenção de realizar uma operação de recepção, retornando também imediatamente, e nada se garante em relação aos dados recebidos já estarem disponíveis para uso pela aplicação do usuário.

Comunicação Assíncrona

- Tanto no envio como na recepção, ainda é necessário uma **segunda fase**, onde é verificado se as operações de troca de mensagem iniciadas anteriormente pelas primitivas de envio e recepção já foram concluídas.
- Somente após essa verificação é possível se alterar com segurança a área de dados utilizada pelo processo emissor e se ter certeza que os dados recebidos estão disponíveis para uso no processo receptor da mensagem.
- O modo de comunicação assíncrono permite uma maior superposição no tempo entre o processamento da aplicação e a transmissão das mensagens, aumentando o desempenho da aplicação paralela.



Escola Supercomputador Santos Dumont - 2026

Avaliação de Desempenho

Avaliação de Desempenho

- Ao desenvolver uma aplicação paralela, é fundamental avaliar seu desempenho com precisão.
- Além de obter tempos de execução melhores que a versão sequencial, é necessário considerar métricas que indiquem se os recursos disponíveis estão sendo utilizados de forma eficiente.
- Em outras palavras:
 - A aplicação paralela está sendo executada de maneira eficiente?
 - É possível melhorar seus tempos de execução?
 - Onde estão os gargalos que impedem essa melhoria?

Avaliação de Desempenho

- A avaliação de desempenho da aplicação paralela, pode ser feita com métricas muito simples:
 - *speedup* – também conhecido como ganho de desempenho ou aceleração;
 - eficiência;
 - escalabilidade.
- Diversos fatores influenciam essas métricas, tais como o algoritmo paralelo utilizado; os custos de sincronização e comunicação; a forma de distribuição das tarefas; e o percentual do programa que efetivamente pode ser paralelizado.

Speed-up (Aceleração)

- O *speedup*, ou aceleração, mede a razão entre o tempo gasto para execução de um algoritmo ou aplicação sequencialmente, com um único processador – $T(1)$, e o tempo gasto na execução com P processadores – $T(P)$, conforme a equação a seguir:

$$S(P) = \frac{T(1)}{T(P)} \quad (2.1)$$

- Por exemplo, se o tempo de execução sequencial $T(1)$ foi 10 segundos, e o tempo de execução paralelo $T(P)$ foi 1 segundo, o *speed-up* $S(P)$ é 10.

Eficiência

- A **eficiência** mede o quão eficaz é a adição de novos processadores para auxiliar na resolução de um problema.
- Seu valor é calculado pela razão entre o speedup $S(P)$ e o número de processadores P utilizados para alcançar esse *speedup*, conforme mostra a equação a seguir:

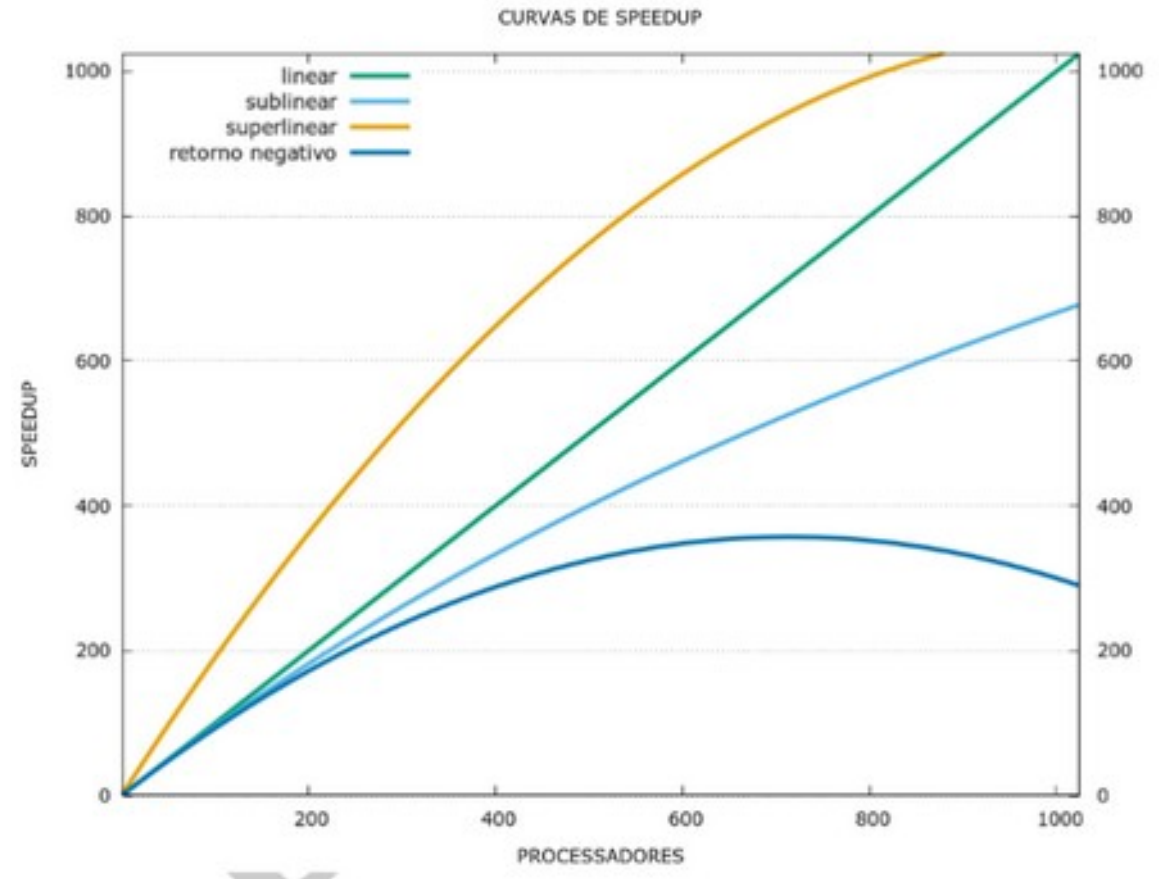
$$E(P) = \frac{S(P)}{P} \quad (2.3)$$

Eficiência

- Então, considerando o exemplo anterior, se o *speedup* igual a 10 foi obtido com 10 processadores, a eficiência será igual a 1, o que é um valor ótimo.
- Contudo, se o mesmo speedup foi obtido com o uso de 20 processadores, então a eficiência é igual a 0,5, o que pode ser considerado um valor baixo.
- Normalmente, o valor da eficiência é menor do que 1 devido aos seguintes fatores, já elencados:
 - O percentual de código paralelizável na aplicação;
 - A distribuição de carga entre os processadores;
 - A quantidade de comunicação e sincronização da aplicação.

Speedup vs Eficiência

- O **speedup** pode apresentar os comportamentos da figura ao lado, de acordo com os valores de eficiência:
 - Superlinear $E \rightarrow E > 1$
 - Linear $\rightarrow E = 1$
 - Sublinear $\rightarrow E < 1$
 - Retorno negativo $\rightarrow E < 0$



Speedup versus Eficiência

- A eficiência pode ser interpretada graficamente como a inclinação da curva de *speedup*.
- Em condições ideais, a inclinação da curva será de 45° , ou seja, o *speedup* será sempre igual a P , onde P indica o número de processadores em uso. Isso caracteriza o *speedup linear*, no qual a eficiência é igual a 1 .
- Entretanto, o *speedup sublinear* é o caso mais comum, ocorrendo devido a ineficiências no processo de paralelização dos programas já mencionados:
 - O percentual de código que pode ser paralelizado na aplicação;
 - A distribuição desigual de carga entre os processadores;
 - A quantidade de comunicação e sincronização necessária na aplicação.

Speedup versus Eficiência

- O *speedup superlinear* ocorre apenas em situações muito específicas, tal como, quando a paralelização altera a distribuição dos dados na hierarquia de memória.
- Isso pode permitir, por exemplo, que os dados de cada processador caibam inteiramente na memória cache, em vez de precisarem ser buscados na memória principal.
- O *retorno negativo* ocorre quando a adição de mais processadores **diminui** o *speedup*, aumentando o tempo de execução da aplicação, possivelmente por conta de um aumento excessivo nos custos de comunicação e/ou de sincronização.

Escalabilidade

- Uma outra métrica importante é a escalabilidade. Ou seja, como o meu programa se comporta conforme aumenta o número de processadores envolvidos na computação.
- Um sistema é considerado **escalável** quando sua eficiência se mantém constante à medida que o número de processadores P aplicados à solução do problema aumenta.
- Quando o tamanho do problema é mantido constante e o número de processadores cresce, dois fenômenos acontecem: o **overhead** de comunicação tende a aumentar, resultando na diminuição da eficiência; e granularidade de cada tarefa executada diminui.

Escalabilidade

- Na prática, uma análise de escalabilidade deve levar em conta a possibilidade de aumentar proporcionalmente o tamanho do problema a ser resolvido conforme P cresce, de modo a compensar o aumento natural da sobrecarga de comunicação e sincronização.
- Considere, por exemplo, um problema de tamanho S . Utilizando P processadores, esse problema leva um tempo T para ser executado. O sistema é considerado escalável se um problema de tamanho $2S$ executado em $2P$ processadores, também levar o mesmo tempo T .
- Assim, se esses cuidados forem tomados, uma análise mais adequada do sistema pode ser realizada.



Escola Supercomputador Santos Dumont - 2026

MPI

MPI

- É um padrão de troca de mensagens portátil que facilita o desenvolvimento de aplicações paralelas.
- Usa o paradigma de programação paralela por troca de mensagens e pode ser usado tanto em *clusters* como em sistemas de memória compartilhada.
- É uma biblioteca de funções utilizável com programas escritos em C, C++ ou Fortran.
- O MPI foi fortemente influenciado pelo trabalho no IBM T. J. Watson Research Center, Intel's NX/2, Express, nCUBE's Vertex e PARMACS. Outras contribuições importantes vieram do Zipcode, Chimp, PVM, Chameleon e PICL.

Características

- Suporte para comunicação entre processos ponto a ponto ou coletiva.
- Suporte para vários modelos de comunicação, incluindo comunicação síncrona e assíncrona.
- Suporte para vários tipos de topologias de rede virtuais, incluindo anel, árvore e malha.
- Suporte para tipos de dados básicos e estruturados.
- Suporte para operações de comunicação coletiva como difusão, redução e dispersão.
- Portabilidade entre diferentes arquiteturas de computação paralela e sistemas operacionais.

Características

- O MPI pode ser combinado com outros modelos de programação.
- Assim, podemos combinar MPI com OpenMP, que apresenta vantagens em alguns casos, como por exemplo:
 - Códigos com escalabilidade MPI limitada, quer seja pelo algoritmo ou pelas rotinas de comunicação coletiva utilizadas.
 - Códigos limitado pelo tamanho de memória, tendo muitos dados replicados em cada processo MPI.
 - Códigos com problemas de desempenho pela ineficiência da implementação da comunicação intra-nó em MPI.
- O MPI também pode ser combinado com OpenMP, OpenACC ou CUDA para uso de aceleradores.

Linguagens Suportadas

- **C, C++ e Fortran:** O MPI foi originalmente desenvolvido para ser usado com o C, C++ e Fortran.
- **Python:** Existem várias bibliotecas MPI para Python, como mpi4py e pyMPI, que permitem que os programas Python se comuniquem usando MPI.
- **Java:** Existe uma biblioteca MPI para Java chamada MPJ Express, que permite que os programas Java se comuniquem usando MPI.
- **Outras linguagens:** Além das linguagens mencionadas acima, também existem bibliotecas MPI para outras linguagens como Perl, Ruby, Lua, entre outras.

Histórico de versões

- MPI-1.1: primeira versão funcional lançada em 1998.
- MPI-1.3: documento final que consolida a versão 1.0 do MPI, lançada apenas em 2008.
- MPI-2.1: lançada oficialmente em 2008 como um livro com diversos exemplos e orientações para os usuários.
- MPI-2.2: lançada em 2009. A última versão desta série.
- MPI-3.1: a versão final do padrão 3.0, foi lançada em 2015.
- MPI 4.0: a versão mais recente do padrão foi lançada em 2021.
- Todas as versões estão disponíveis em <https://www.mpi-forum.org/docs/>

Objetivos

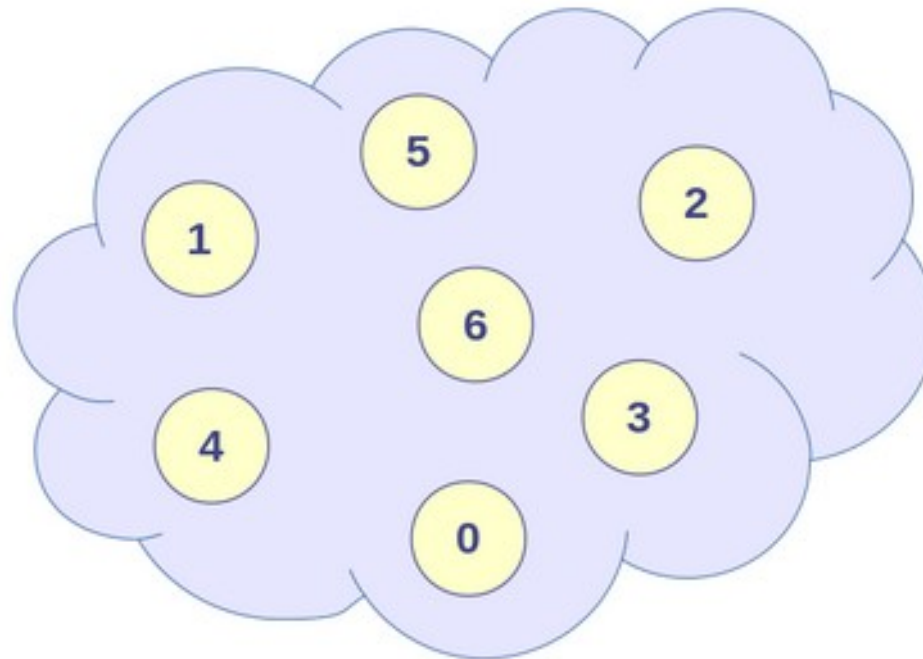
- Um dos objetivos do MPI é oferecer possibilidade de uma implementação eficiente da comunicação:
 - Evitando cópias de memória para memória.
 - Permitindo superposição de comunicação e computação.
- Permitir implementações em ambientes heterogêneos.
- Supõe-se que a interface de comunicação é confiável:
 - Falhas de comunicação devem ser tratadas pelo subsistema de comunicação da plataforma.

Comunicadores

- A biblioteca MPI trabalha com o conceito de comunicadores para definir o universo de processos envolvidos em uma operação de comunicação, através dos atributos de grupo e contexto:
 - Dois processos que pertencem a um mesmo grupo e usando um mesmo contexto podem se comunicar diretamente.
 - O comunicador padrão recebe o nome de `MPI_COMM_WORLD` e contém todos os processos que iniciados na execução de um programa.
 - Cada processo possui um identificador único chamado de `ranque` (rank), que vai de 0 até $P-1$, onde P é o número de processos em um comunicador.

Comunicadores

Comunicador



Comunicadores

- Comunicadores são utilizados para definir o universo de processos envolvidos em uma operação de comunicação através dos seguintes atributos:
 - **Grupo:** Conjunto ordenado de processos. A ordem de um processo no grupo é chamada de ranque.
 - **Contexto:** tag definido pelo sistema.
- Dois processos pertencentes a um mesmo grupo e usando um mesmo contexto podem se comunicar.
- O comunicador padrão **MPI_COMM_WORLD** contém todos os processos existentes no início da execução de um programa.



Escola Supercomputador Santos Dumont - 2026

Programação com MPI

Ementa

- Exemplo de um programa MPI
- Rotinas básicas de gerenciamento.
- Rotinas básicas de comunicação ponto a ponto
- Correspondência entre os tipos MPI e C
- Identificando o tamanho e as informações da mensagem recebida

Bibliografia



<https://www.casadocodigo.com.br/products/livro-programacao-paralela>



Escola Supercomputador Santos Dumont - 2026

MPI - Introdução

Pré-requisitos

- Para os exemplos apresentados neste curso, será necessário um sistema operacional do tipo Linux.
- Preferencialmente, você pode utilizar as distribuições Fedora, Ubuntu ou WSL (no Windows).
- Mas os exemplos e comandos podem ser utilizados em outras distribuições desde que os pacotes necessários estejam corretamente instalados.
- Certifique-se de ter um compilador instalado. O MPI geralmente funciona com compiladores como gcc, g++, ou gfortran.

Pré-requisitos

- Instale uma implementação do MPI, as mais comuns são:
 - MPICH: Uma implementação amplamente usada.
 - OpenMPI: Populares em sistemas de alto desempenho.
- Em distribuições Linux do tipo Debian, você pode instalar usando:
`$ sudo apt install mpich # MPICH`
`$ sudo apt install openmpi-bin openmpi-common # OpenMPI`
- Os exemplos apresentados neste curso estão disponíveis no repositório <https://github.com/Programacao-Paralela-e-Distribuida/MPI>

Compilando MPI

- De uma maneira simplificada, um programa em MPI pode ser compilado e executado com os seguintes comandos em um terminal de um sistema operacional do tipo Linux utilizando-se o compilador gcc:

```
$ mpicc -o teste mpi_simples.c
```

```
$ mpirun -np 4 ./teste
```

- Onde **np** indica o número de processos com que o programa deve ser executado.
- A opção **-hostfile** define quais máquinas utilizar em um cluster.

Google Colab

- Os exemplos, notebooks e slides do minicurso estão disponíveis no seguinte endereço:
- <https://github.com/Programacao-Paralela-e-Distribuida/MPI/>

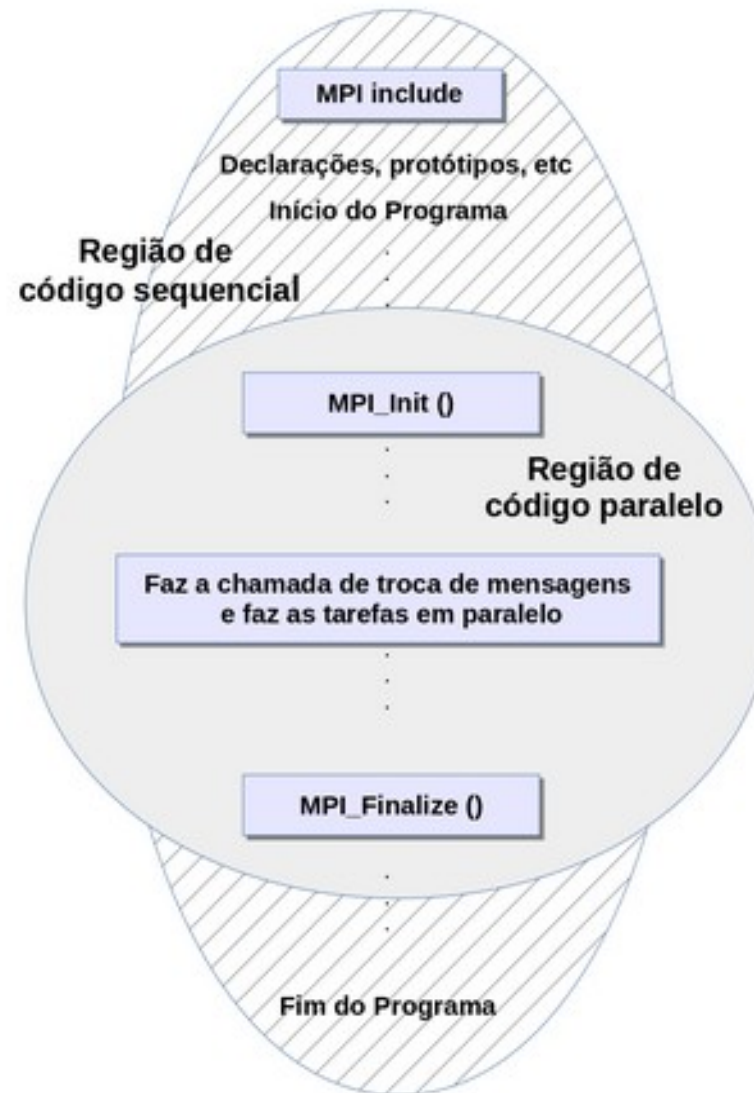
Programando com MPI

- Todo programa em MPI deve conter a seguinte diretiva para o pré-processador:
`#include "mpi.h"`
- Este arquivo, `mpi.h`, contém as definições, macros e funções de protótipos de funções necessários para a compilação de um programa MPI.
- Antes de qualquer outra função MPI ser chamada, a função `MPI_Init()` deve ser chamada pelo menos uma vez.
- Seus argumentos são os ponteiros para os parâmetros do programa principal, `argc` e `argv`.

Programando com MPI

- A função `MPI_Init()` permite que o sistema realize as operações de preparação necessárias para que a biblioteca MPI seja utilizada.
- Ao término do programa a função `MPI_Finalize()` deve ser chamada.
- Esta função limpa qualquer pendência deixada pelo MPI, p. ex, recepções pendentes que nunca foram completadas.
- Tipicamente, um programa em MPI deve ter o seguinte leiaute:

Programa MPI



Programa MPI

```
...
#include "mpi.h"
...
main(int argc, char** argv) {
...
/* Nenhuma função MPI pode ser chamada antes deste ponto */
MPI_Init(&argc, &argv);
...
MPI_Finalize();
/* Nenhuma função MPI pode ser chamada depois deste ponto*/
...
}
```


Quem sou eu?

- O MPI tem a função `MPI_Comm_Rank()` que retorna o **ranque** (rank) de um processo no seu segundo argumento.
- Sua sintaxe é:

```
int MPI_Comm_rank(MPI_Comm com, int *ranque)
```
- O primeiro argumento é um **comunicador**. Essencialmente um comunicador é uma coleção de processos que podem enviar mensagens entre si.
- Para os programas básicos, o único comunicador necessário é `MPI_COMM_WORLD`, que é pré-definido no MPI e consiste de todos os processos executando quando a execução do programa começa.

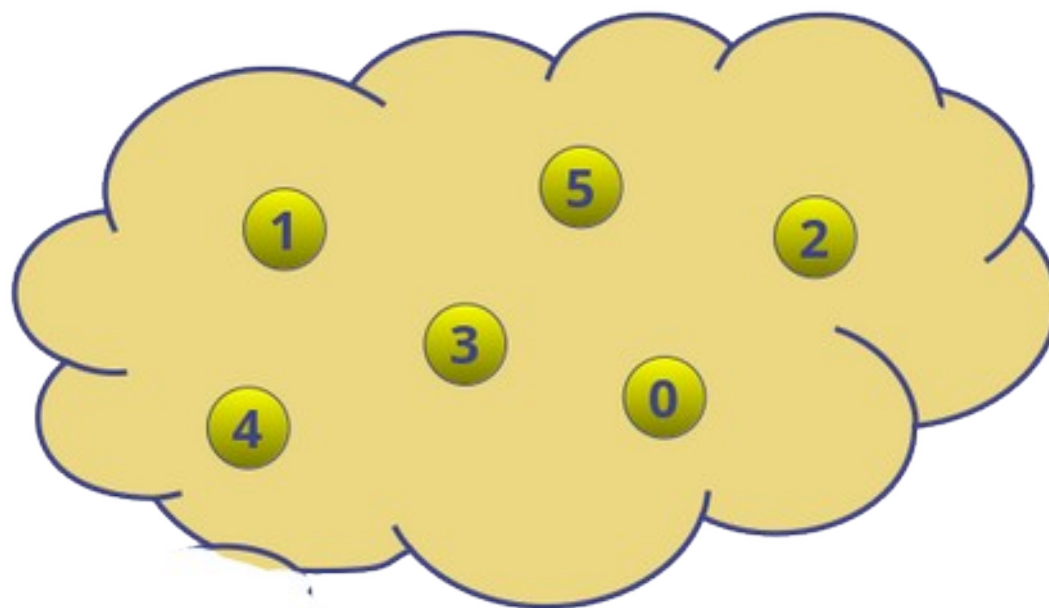
Quantos processos somos?

- Muitas construções em nossos programas também dependem do número de processos executando o programa.
- O MPI oferece a função `MPI_Comm_size()` para determinar este valor.
- Essa função retorna o número de processos em um comunicador no seu segundo argumento.
- Sua sintaxe é:

```
int MPI_Comm_size(MPI_Comm com, int *num_procs)
```

Ranke

MPI_COMM_WORLD



Funções Básicas

- Abortando um programa:

`int MPI_Abort(MPI_Comm com, int erro)`

- Identificando a versão do MPI:

`int MPI_Get_version(int *versao, int *subversao)`

- Recuperando o nome do computador:

`int MPI_Get_processor_name (char *nome, int *comprimento)`

Funções Básicas

```
#include <stdio.h>
#include "mpi.h"
int main(int argc, char *argv[]) {    /* mpi_funcoes.c */
    int meu_rank, num_procs;
    int versao, subversao, aux, ret;
    char maquina[MPI_MAX_PROCESSOR_NAME];
    /* Inicia o MPI. Em caso de erro aborta o programa */
    ret = MPI_Init(&argc, &argv);
    if (ret != MPI_SUCCESS) {
        printf("Erro ao iniciar o programa MPI. Abortando.\n");
        MPI_Abort(MPI_COMM_WORLD, ret);
    }
    /* Imprime a versão e subversão da biblioteca MPI */
    MPI_Get_version(&versao, &subversao);
    printf("Versão do MPI = %d Subversão = %d \n", versao, subversao);
    /* Obtém o rank e número de processos em execução */
    MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
    MPI_Comm_rank(MPI_COMM_WORLD, &meu_rank);
    /* Define o nome do computador onde o processo está executando */
    MPI_Get_processor_name(maquina, &aux);
    printf("Número de tarefas = %d Meu rank = %d Executando em %s\n", num_procs, meu_rank, maquina);
    /* Finaliza o MPI */
    MPI_Finalize();
    return(0);
}
```

Funções Básicas

```
$ mpicc -o teste mpi_funcoes.c
```

```
$ mpirun -n 2 ./teste
```

Versão do MPI = 3 Subversão = 1

Versão do MPI = 3 Subversão = 1

Número de tarefas = 2 Meu ranque = 1 Executando em Incc-2025

Foram gastos 0.000100 segundos com precisão de 1.000e-09 segundos

Número de tarefas = 2 Meu ranque = 0 Executando em Incc-2025

Foram gastos 0.000135 segundos com precisão de 1.000e-09 segundos

Medindo o tempo de execução

- A função `MPI_Wtime()` retorna (em precisão dupla) o tempo total em segundos decorrido desde um instante determinado no passado.
- Esse instante é dependente de implementação, mas deve sempre o mesmo para uma dada implementação.
- A função `MPI_Wtick()` retorna (em precisão dupla) a resolução em segundos da função `MPI_Wtime()`.
- Um exemplo de uso dessas funções pode ser visto a seguir.

Escola de Computação Santos Dumont - 2026

```
#include "mpi.h"
#include <stdio.h>
int main(int argc, char *argv[ ]) {    /* mpi_wtime.c */
double tempo_inicial, tempo_final, a;
    tempo_inicial = MPI_Wtime();
    for(long int i = 0; i < 1000000000000; i++) {
        a = (double) i;    /* Realiza o trabalho em paralelo */
    }
    tempo_final = MPI_Wtime();
    printf("Foram gastos %3.6f segundos para calcular a = %3.0f com \
precisão de %3.3e segundos\n", tempo_final-tempo_inicial, a, MPI_Wtick ());
    return(0);
}
```


Medindo o tempo de execução

```
$ mpicc -o teste mpi_wtime.c
```

```
$ mpirun -n 2 ./teste
```

Foram gastos 19.818745 segundos para calcular $a = 9999999999$ com precisão de $1.000e-09$ segundos

Foram gastos 19.900254 segundos para calcular $a = 9999999999$ com precisão de $1.000e-09$ segundos



Escola Supercomputador Santos Dumont - 2026

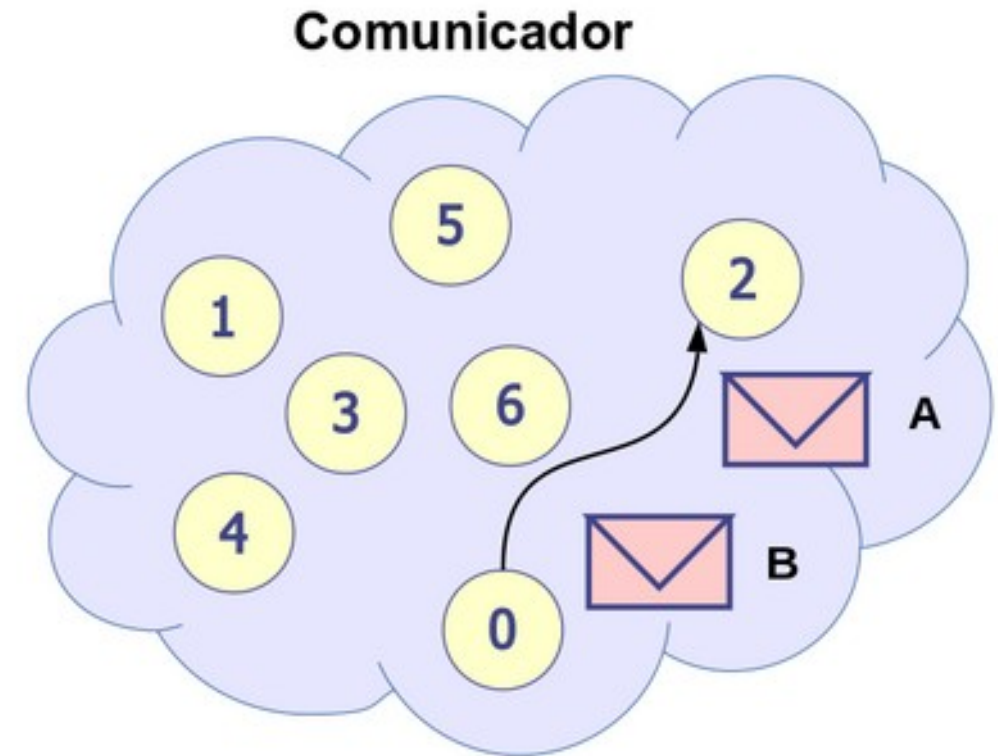
Comunicação Ponto a Ponto

Mensagem MPI

- Mensagem = Dados + Envelope
- Para que a mensagem seja comunicada com sucesso, o sistema deve anexar alguma informação aos dados que o programa de aplicação deseja transmitir.
- Essa informação adicional forma o envelope da mensagem, que no MPI contém a seguinte informação:
 - O **ranque** do processo origem.
 - O **ranque** do processo destino.
 - Uma **etiqueta** especificando o tipo da mensagem.
 - Um **comunicador** definindo o domínio de comunicação.

Ordem das Mensagens

- As mensagens não ultrapassam umas às outras.
- Por exemplo, se o processo com ranque 0 enviar duas mensagens sucessivas A e B, e o processo com ranque 2 chamar duas rotinas de recepção que combinam com qualquer uma das mensagens, a ordem das mensagens é preservada, sendo que A será sempre recebida antes de B.



Comunicação Ponto-a-Ponto

- O mecanismo de troca de mensagens ponto-a-ponto no MPI é realizado pelas funções `MPI_Send()` e `MPI_Recv()`.
- Quando dois processos estão se comunicando utilizando `MPI_Send()` e `MPI_Recv()`, a importância do uso do **comunicador** aumenta quando módulos de um programa são desenvolvidos de forma independente.
- Por exemplo, ao desenvolver uma biblioteca para resolver um sistema de equações lineares, você pode criar um comunicador exclusivo para o solucionador linear.
- Isso evita conflitos entre as mensagens, mesmo que etiquetas idênticas sejam utilizadas em outros módulos.

Comunicação Ponto-a-Ponto

- Vamos utilizar por enquanto o comunicador pré-definido `MPI_COMM_WORLD`, que inclui todos os processos ativos desde o início da execução do programa.
- A rotina `MPI_Send()` envia a mensagem para um determinado processo e a rotina `MPI_Recv()` recebe a mensagem de um processo.
- Ambas são operações **bloqueantes**:
 - Envio bloqueante: aguarda até que a mensagem seja completamente copiada das variáveis do programa do usuário para os buffers de envio, seja do sistema operacional ou da biblioteca MPI.
 - Recepção bloqueante: aguarda até que a mensagem recebida esteja totalmente armazenada nas variáveis do programa do usuário e pronta para uso.

Comunicação Ponto-a-Ponto

- A maioria das funções MPI é do tipo inteiro e retorna um código de erro ao final da chamada.
- Contudo, como a maioria dos programadores em 'C', ao longo deste curso, nós vamos ignorar este código em quase todos exemplos apresentados.
- Apesar disso, em aplicações profissionais, é recomendável implementar o tratamento de erro das funções para identificar problemas durante a execução dos programas.

MPI_Send()

```
int MPI_Send(void* mensagem, int cont, MPI_Datatype tipo_mpi, int destino,  
int etiq, MPI_Comm com)
```

- **mensagem**: endereço inicial da mensagem a ser enviada.
- **cont**: número de elementos da mensagem.
- **tipo_mpi**: tipo de dados da mensagem.
- **destino**: ranque do processo destino da mensagem.
- **etiq**: etiqueta da mensagem.
- **com**: comunicador do contexto da mensagem.

MPI_Send()

- A mensagem a ser enviada está armazenada em uma posição de memória definida pelo ponteiro `*mensagem`.
- A mensagem é um vetor com `cont` elementos do mesmo tipo, especificado pelo argumento `tipo_mpi`.
- Esses dois parâmetros, combinados, permitem que o sistema determine corretamente o tamanho da mensagem a ser enviada.
- Para evitar ambiguidades entre arquiteturas diferentes, o MPI define uma lista de tipos pré-definidos, tais como `MPI_CHAR`, `MPI_INT`, `MPI_FLOAT`, `MPI_BYTE`, `MPI_LONG`, `MPI_UNSIGNED_CHAR`, etc.

MPI_Send()

- O argumento **destino** é o ranque do processo para o qual a mensagem será enviada. Não há um “coringa” para o destino; ele deve sempre ser definido de forma única e explícita.
- O **etiq** é um inteiro usado pelo processo de destino para diferenciar mensagens distintas enviadas pelo mesmo processo de origem.
- O parâmetro **com** define, por meio do comunicador, o contexto da comunicação e os processos participantes do grupo. O comunicador padrão é **MPI_COMM_WORLD** e, por enquanto, será o único comunicador que utilizaremos.

MPI_Recv()

`int MPI_Recv(void* mensagem, int cont, MPI_Datatype tipo_mpi, int origem, int etiq, MPI_Comm com, MPI_Status* estado)`

- **mensagem:** endereço inicial onde a mensagem vai ser armazenada.
- **cont:** número máximo de elementos a serem recebidos.
- **tipo_mpi:** tipo de dados esperado na mensagem.
- **origem:** ranque do processo origem.
- **etiq:** etiqueta esperada da mensagem.
- **com:** comunicador do contexto da mensagem.
- **estado:** estrutura auxiliar com informações como o remetente e a etiqueta da mensagem.

MPI_Recv()

- A mensagem recebida vai ser armazenada em uma posição de memória definida pelo ponteiro `*mensagem`.
- A mensagem recebida poderá ter, no máximo, `cont` elementos do do tipo `tipo_mpi`.
- O argumento `origem` é o ranque do processo do qual estamos esperando a mensagem.
- O MPI permite que `origem` seja um coringa (*), neste caso usamos `MPI_ANY_SOURCE` neste parâmetro.
- Reforçamos que não há um “coringa” para o destino da mensagem.

MPI_Recv()

- `etiq` é especificado pelo usuário para distinguir as mensagens de um único processo.
- O MPI garante que inteiros entre 0 e 32767 possam ser usados como etiquetas, mas o valor máximo é dependente de implementação.
- Existe um coringa, `MPI_ANY_TAG`, que a função `MPI_Recv` pode usar como etiqueta.
- O último argumento de `MPI_Recv()`, chamado de `estado`, fornece informações detalhadas sobre os dados efetivamente recebidos.

Comunicação ponto-a-ponto

- Esse argumento corresponde a uma estrutura que inclui os seguintes campos principais:
 - **MPI_SOURCE**: indica o ranque do processo remetente.
 - **MPI_TAG**: identifica a etiqueta da mensagem recebida.
 - **MPI_ERROR**: Informa se a mensagem foi recebida corretamente ou não.
- Por exemplo, se a origem da recepção foi especificada como **MPI_ANY_SOURCE**, o campo **MPI_SOURCE** conterá o ranque do processo que enviou a mensagem.
- Se a etiqueta de recepção foi especificada como **MPI_ANY_TAG**, o campo **MPI_TAG** contém a etiqueta da mensagem recebida.

Comunicação ponto-a-ponto

- Para que a comunicação entre dois processos **A** e **B** no MPI ocorra corretamente, os argumentos usados por `MPI_Send()` no processo **A** devem ser rigorosamente compatíveis com os usados por `MPI_Recv()` no processo **B**.
- Isso ocorre porque o MPI utiliza esses argumentos para estabelecer uma correspondência entre o envio e o recebimento das mensagens.
- Qualquer incompatibilidade pode resultar em erros, mensagens perdidas ou comportamentos inesperados.
- Em primeiro lugar, o mesmo comunicador deve ser usado em ambos processos para que a mensagem seja transferida corretamente.

Comunicação ponto-a-ponto

- No processo **A**, o argumento **destino** em **MPI_Send()** deve corresponder ao ranque do processo **B** (identificador único no comunicador).
- No processo **B**, o argumento **origem** em **MPI_Recv()** deve ser o ranque do processo **A** ou um coringa, como **MPI_ANY_SOURCE**.
- Ambos os processos devem usar a mesma etiqueta para identificar a mensagem.
- A etiqueta permite que um processo receba mensagens específicas, mesmo quando há múltiplas mensagens em trânsito.
- Se o coringa **MPI_ANY_TAG** for usado no processo **B**, qualquer etiqueta será aceita.

Exemplo

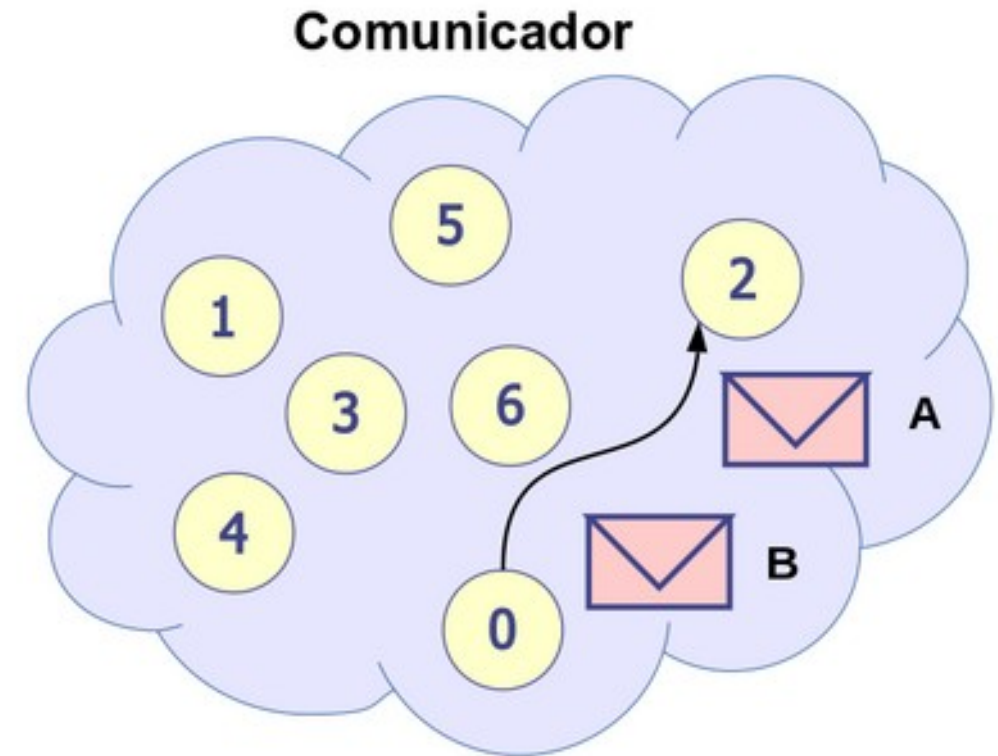
```
#include <stdio.h>
#include <string.h>
#include "mpi.h"
main(int argc, char** argv) {
    int meu_ranque, num_procs, origem, destino, etiq=0;
    char mensagem[100];
    MPI_Status estado;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &meu_ranque);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
    if (meu_ranque != 0){
        sprintf(msg, "Processo %d está vivo!", meu_ranque);
        destino = 0;
        MPI_Send(mensagem, strlen(mensagem)+1, MPI_CHAR, destino, etiq, MPI_COMM_WORLD);
    }
    else{
        for (origem=1; origem < num_procs; origem++) {
            MPI_Recv(mensagem, 100, MPI_CHAR, origem, etiq, MPI_COMM_WORLD, &estado);
            printf("%s\n", mensagem);
        }
    }
    MPI_Finalize( );
}
```

Mensagem MPI

- Mensagem = Dados + Envelope
- Para que a mensagem seja comunicada com sucesso, o sistema deve anexar alguma informação aos dados que o programa de aplicação deseja transmitir.
- Essa informação adicional forma o envelope da mensagem, que no MPI contém a seguinte informação:
 - O **ranque** do processo origem.
 - O **ranque** do processo destino.
 - Uma **etiqueta** especificando o tipo da mensagem.
 - Um **comunicador** definindo o domínio de comunicação.

Ordem das Mensagens

- As mensagens não ultrapassam umas às outras.
- Por exemplo, se o processo com ranque 0 enviar duas mensagens sucessivas A e B, e o processo com ranque 2 chamar duas rotinas de recepção que combinam com qualquer uma das mensagens, a ordem das mensagens é preservada, sendo que A será sempre recebida antes de B.



Comunicação Ponto-a-Ponto

- O mecanismo de troca de mensagens ponto-a-ponto no MPI é realizado pelas funções `MPI_Send()` e `MPI_Recv()`.
- Quando dois processos estão se comunicando utilizando `MPI_Send()` e `MPI_Recv()`, a importância do uso do comunicador aumenta quando módulos de um programa são desenvolvidos de forma independente.
- Por exemplo, ao desenvolver uma biblioteca para resolver um sistema de equações lineares, você pode criar um comunicador exclusivo para o solucionador linear.
- Isso evita conflitos entre as mensagens, mesmo que etiquetas idênticas sejam utilizadas em outros módulos.

Comunicação Ponto-a-Ponto

- Vamos utilizar por enquanto o comunicador pré-definido `MPI_COMM_WORLD`, que inclui todos os processos ativos desde o início da execução do programa.
- A rotina `MPI_Send()` envia a mensagem para um determinado processo e a rotina `MPI_Recv()` recebe a mensagem de um processo.
- Ambas são operações **bloqueantes**:
 - Envio bloqueante: aguarda até que a mensagem seja completamente copiada das variáveis do programa do usuário para os buffers de envio, seja do sistema operacional ou da biblioteca MPI.
 - Recepção bloqueante: aguarda até que a mensagem recebida esteja totalmente armazenada nas variáveis do programa do usuário e pronta para uso.

Comunicação Ponto-a-Ponto

- A maioria das funções MPI é do tipo inteiro e retorna um código de erro ao final da chamada.
- Contudo, como a maioria dos programadores em 'C', ao longo deste curso, nós vamos ignorar este código em quase todos exemplos apresentados.
- Apesar disso, em aplicações profissionais, é recomendável implementar o tratamento de erro das funções para identificar problemas durante a execução dos programas.

MPI_Send()

```
int MPI_Send(void* mensagem, int cont, MPI_Datatype tipo_mpi, int destino,  
int etiq, MPI_Comm com)
```

- **mensagem**: endereço inicial da mensagem a ser enviada.
- **cont**: número de elementos da mensagem.
- **tipo_mpi**: tipo de dados da mensagem.
- **destino**: ranque do processo destino da mensagem.
- **etiq**: etiqueta da mensagem.
- **com**: comunicador do contexto da mensagem.

MPI_Send()

- A mensagem a ser enviada está armazenada em uma posição de memória definida pelo ponteiro `*mensagem`.
- A mensagem é um vetor com `cont` elementos do mesmo tipo, especificado pelo argumento `tipo_mpi`.
- Esses dois parâmetros, combinados, permitem que o sistema determine corretamente o tamanho da mensagem a ser enviada.
- Para evitar ambiguidades entre arquiteturas diferentes, o MPI define uma lista de tipos pré-definidos, tais como MPI_CHAR, MPI_INT, MPI_FLOAT, MPI_BYTE, MPI_LONG, MPI_UNSIGNED_CHAR, etc.

MPI_Send()

- O argumento **destino** é o ranque do processo para o qual a mensagem será enviada. Não há um “coringa” para o destino; ele deve sempre ser definido de forma única e explícita.
- O **etiq** é um inteiro usado pelo processo de destino para diferenciar mensagens distintas enviadas pelo mesmo processo de origem.
- O parâmetro **com** define, por meio do comunicador, o contexto da comunicação e os processos participantes do grupo. O comunicador padrão é **MPI_COMM_WORLD** e, por enquanto, será o único comunicador que utilizaremos.

MPI_Recv()

`int MPI_Recv(void* mensagem, int cont, MPI_Datatype tipo_mpi, int origem, int etiq, MPI_Comm com, MPI_Status* estado)`

- **mensagem:** endereço inicial onde a mensagem vai ser armazenada.
- **cont:** número máximo de elementos a serem recebidos.
- **tipo_mpi:** tipo de dados esperado na mensagem.
- **origem:** ranque do processo origem.
- **etiq:** etiqueta esperada da mensagem.
- **com:** comunicador do contexto da mensagem.
- **estado:** estrutura auxiliar com informações como o remetente e a etiqueta da mensagem.

MPI_Recv()

- A mensagem recebida vai ser armazenada em uma posição de memória definida pelo ponteiro `*mensagem`.
- A mensagem recebida poderá ter, no máximo, `cont` elementos do do tipo `tipo_mpi`.
- O argumento `origem` é o ranque do processo do qual estamos esperando a mensagem.
- O MPI permite que `origem` seja um coringa (*), neste caso usamos `MPI_ANY_SOURCE` neste parâmetro.
- Reforçamos que não há um “coringa” para o destino da mensagem.

MPI_Recv()

- `etiq` é especificado pelo usuário para distinguir as mensagens de um único processo.
- O MPI garante que inteiros entre 0 e 32767 possam ser usados como etiquetas, mas o valor máximo é dependente de implementação.
- Existe um coringa, `MPI_ANY_TAG`, que a função `MPI_Recv` pode usar como etiqueta.
- O último argumento de `MPI_Recv()`, chamado de `estado`, fornece informações detalhadas sobre os dados efetivamente recebidos.

Comunicação ponto-a-ponto

- Esse argumento corresponde a uma estrutura que inclui os seguintes campos principais:
 - **MPI_SOURCE**: indica o ranque do processo remetente.
 - **MPI_TAG**: identifica a etiqueta da mensagem recebida.
 - **MPI_ERROR**: Informa se a mensagem foi recebida corretamente ou não.
- Por exemplo, se a origem da recepção foi especificada como **MPI_ANY_SOURCE**, o campo **MPI_SOURCE** conterá o ranque do processo que enviou a mensagem.
- Se a etiqueta de recepção foi especificada como **MPI_ANY_TAG**, o campo **MPI_TAG** contém a etiqueta da mensagem recebida.

Comunicação ponto-a-ponto

- Para que a comunicação entre dois processos **A** e **B** no MPI ocorra corretamente, os argumentos usados por `MPI_Send()` no processo **A** devem ser rigorosamente compatíveis com os usados por `MPI_Recv()` no processo **B**.
- Isso ocorre porque o MPI utiliza esses argumentos para estabelecer uma correspondência entre o envio e o recebimento das mensagens.
- Qualquer incompatibilidade pode resultar em erros, mensagens perdidas ou comportamentos inesperados.
- Em primeiro lugar, o mesmo comunicador deve ser usado em ambos processos para que a mensagem seja transferida corretamente.

Comunicação ponto-a-ponto

- No processo **A**, o argumento **destino** em **MPI_Send()** deve corresponder ao ranque do processo **B** (identificador único no comunicador).
- No processo **B**, o argumento **origem** em **MPI_Recv()** deve ser o ranque do processo **A** ou um coringa, como **MPI_ANY_SOURCE**.
- Ambos os processos devem usar a mesma etiqueta para identificar a mensagem.
- A etiqueta permite que um processo receba mensagens específicas, mesmo quando há múltiplas mensagens em trânsito.
- Se o coringa **MPI_ANY_TAG** for usado no processo **B**, qualquer etiqueta será aceita.

Exemplo

```
#include <stdio.h>
#include <string.h>
#include "mpi.h"
main(int argc, char** argv) {
    int meu_ranque, num_procs, origem, destino, etiq=0;
    char mensagem[100];
    MPI_Status estado;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &meu_ranque);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
    if (meu_ranque != 0){
        sprintf(msg, "Processo %d está vivo!", meu_ranque);
        destino = 0;
        MPI_Send(mensagem, strlen(mensagem)+1, MPI_CHAR, destino, etiq, MPI_COMM_WORLD);
    }
    else{
        for (origem=1; origem < num_procs; origem++) {
            MPI_Recv(mensagem, 100, MPI_CHAR, origem, etiq, MPI_COMM_WORLD, &estado);
            printf("%s\n", mensagem);
        }
    }
    MPI_Finalize( );
}
```




Escola Supercomputador Santos Dumont - 2026

Correspondência tipos MPI e C

Correspondência entre tipos MPI e C

Tipo de dados C	Tipo de dado MPI	Tipo de
char	MPI_CHAR	signed
short int	MPI_SHORT	signed
int	MPI_INT	signed
long int	MPI_LONG	signed
unsigned char	MPI_UNSIGNED CHAR	
unsigned short int	MPI_UNSIGNED SHORT	
unsigned int	MPI_UNSIGNED	
unsigned long int	MPI_UNSIGNED LONG	
	MPI_FLOAT	float
	MPI_DOUBLE	double
double	MPI_LONG DOUBLE	long
	MPI_BYTE, MPI_PACKED	

Correspondência entre tipos MPI e C

- Os dois últimos tipos, `MPI_BYTE` e `MPI_PACKED`, não possuem correspondência direta com os tipos padronizados em C.
- O tipo `MPI_BYTE` é útil quando não se deseja realizar conversões entre tipos de dados diferentes.
- Já o tipo `MPI_PACKED` é empregado no envio de mensagens empacotadas.
- É importante observar que a quantidade de espaço alocado no *buffer* de recepção não precisa ser idêntica ao tamanho da mensagem recebida.
- O MPI permite que uma mensagem seja recebida enquanto houver espaço suficiente disponível no *buffer*.

Comunicação ponto-a-ponto

- A informação sobre a recepção de mensagem com o uso de um coringa é retornada pela função `MPI_Recv` em uma estrutura do tipo “`MPI_Status`”.
- Essa estrutura tem diversos usos, por exemplo, para saber o total de elementos recebidos utilize a rotina:

```
int MPI_Get_count(MPI_Status *status,  
MPI_Datatype datatype, int *count)
```

Informação	C
remetente	status.MPI_SOURCE
etiqueta	status.MPI_TAG
erro	status.MPI_ERROR

Exemplo

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"
#define MAX 100
int main(int argc, char *argv[]) { /* mpi_status.c */
    int meu_ranke, total_num, etiq = 0;
    int origem = 0, destino = 1, numeros[MAX];
    MPI_Status estado;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &meu_ranke);
    MPI_Comm_rank(MPI_COMM_WORLD, &meu_ranke);
    if (meu_ranke == origem) {
        /* Escolhe uma quantidade aleatória de inteiros para enviar para o processo 1 */
        srand(MPI_Wtime());
        total_num = (rand() / (float)RAND_MAX) * MAX;
        /* Envia a quantidade de inteiros para o processo 1 */
        MPI_Send(numeros, total_num, MPI_INT, destino, etiq, MPI_COMM_WORLD);
        printf("Processo %d enviou %d números para 1\n", origem, total_num);
    }
}
```

Exemplo

```
else {  
    /* Recebe no máximo MAX números do processo 0 */  
    MPI_Recv(numeros, MAX, MPI_INT, origem, etiq, MPI_COMM_WORLD, &estado);  
    /* Quando chega a mensagem, verifica o status para saber quantos números foram recebidos */  
    MPI_Get_count(&estado, MPI_INT, &total_num);  
    /* Imprime a quantidade de números e a informação adicional que está no manipulador "estado" */  
    printf("Processo %d recebeu %d números. Origem da mensagem = %d, etiqueta = %d\n", destino, \  
        total_num, estado.MPI_SOURCE, estado.MPI_TAG);  
}  
MPI_Finalize();  
return(0);  
}
```

Escola de Computação Santos Dumont - 2026

Exemplo

```
$ mpirun -np 2 ./teste
```

Processo 0 enviou 93 números para 1

Processo 1 recebeu 93 números. Origem da mensagem = 0, etiqueta = 0



Escola Supercomputador Santos Dumont - 2026

Programação com MPI

Ementa

- Rotinas de Comunicação Coletivas
- Barreira: MPI_Barrier()
- Difusão: MPI_Broadcast()
- Coleta: MPI_Gather()
- Redução: MPI_Reduce()
- Redução com difusão: MPI_Allreduce()
- Coleta com difusão: MPI_Allgather()

Bibliografia



<https://www.casadocodigo.com.br/products/livro-programacao-paralela>



Escola Supercomputador Santos Dumont - 2026

Comunicação Coletiva

Escola de Computação Santos Dumont - 2026

Google Colab

- Os exemplos, notebooks e slides do minicurso estão disponíveis no seguinte endereço:
- <https://github.com/Programacao-Paralela-e-Distribuida/MPI/>

Comunicação coletiva

- As operações de comunicação coletiva são mais restritivas que as comunicações ponto a ponto:
 - A quantidade de dados enviados deve casar exatamente com a quantidade de dados especificada pelo receptor.
 - Apenas a versão bloqueante das funções está disponível.
 - O argumento tag não existe.
 - Todos os processos participantes da comunicação coletiva chamam a mesma função com argumentos compatíveis.
 - As funções estão disponíveis apenas no modo padrão*.

* Nas últimas versões do padrão MPI já existem versões não bloqueantes das rotinas de comunicação coletiva.

Comunicação Coletiva

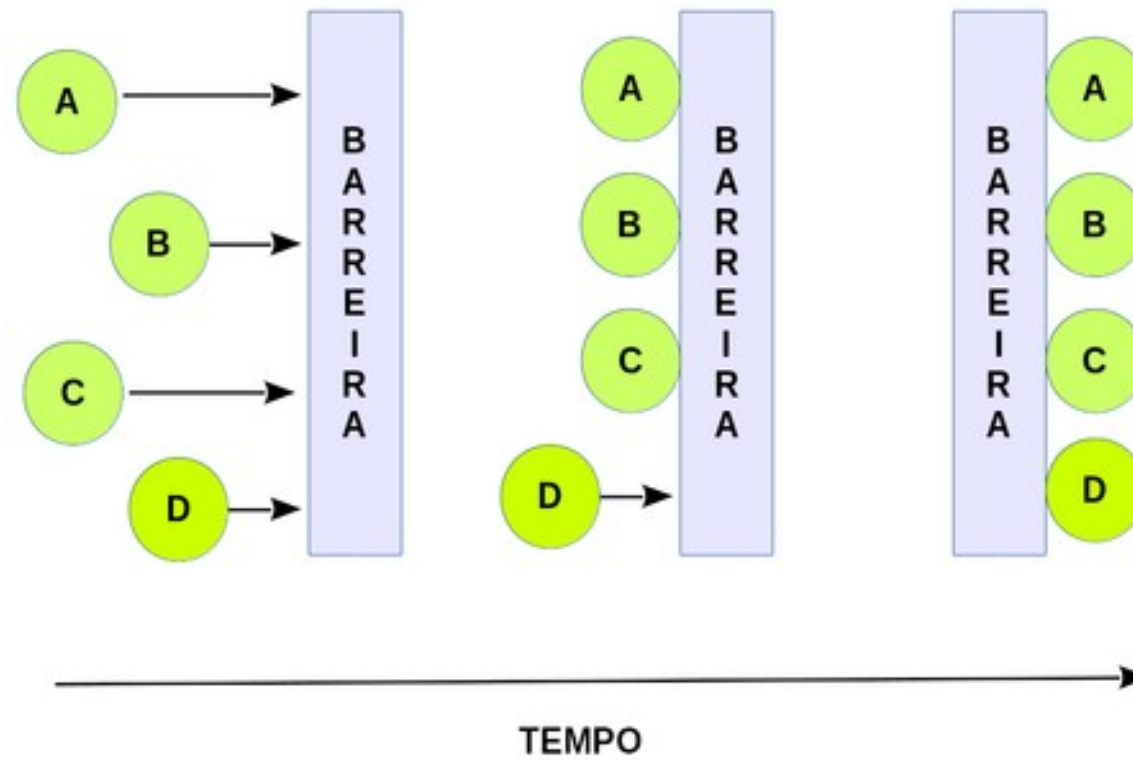
- Quando, em uma operação coletiva, houver um único processo de origem ou um único processo de destino, este processo é chamado de raiz.
- **Barreira**: bloqueia todos os processos até que todos processos do grupo chamem a função.
- **Difusão (broadcast)**: envia a mesma mensagem para todos os processos.
- **Coleta (gather)**: os dados são recolhidos de todos os processos em um único processo.
- **Dispersão (scatter)**: os dados são distribuídos de um processo para os demais.

Comunicação Coletiva

- **Coleta com difusão (Allgather):** um coleta (*gather*) seguida de uma difusão.
- **Redução (reduce):** realiza as operações coletivas de soma, máximo, mínimo, etc.
- **Redução com difusão (Allreduce):** uma redução seguida de uma difusão.
- **Alltoall:** conjunto de coletas (*gathers*) onde cada processo recebe dados diferentes.*

* Não vamos abordar neste nosso estudo

Barreira



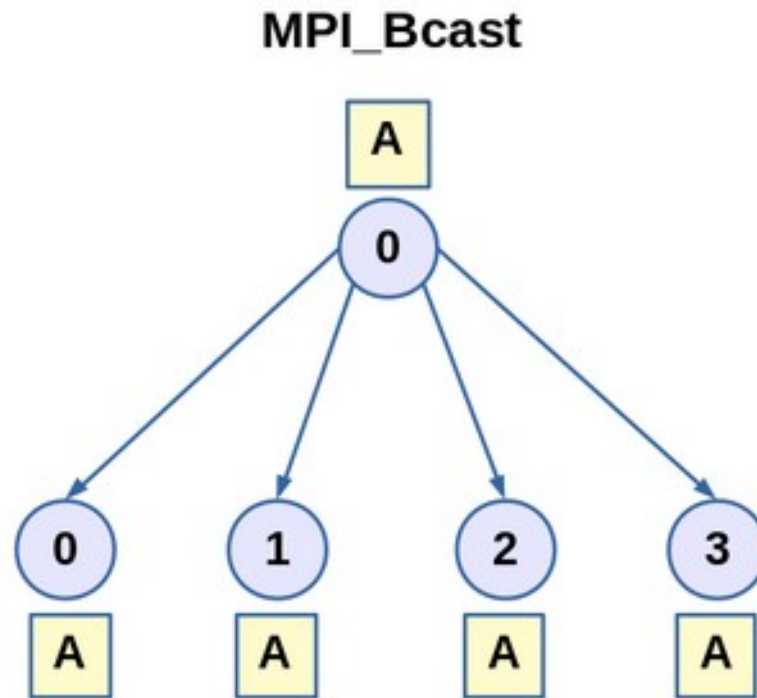
MPI_Barrier()

- Sintaxe:

`int MPI_Barrier(MPI_Comm com)`

- A função `MPI_Barrier()` fornece um mecanismo para sincronizar todos os processos no comunicador `com`.
- Cada processo bloqueia (i.e., pára) até todos os processos no comunicador `com` tenham chamado `MPI_Barrier()`.

MPI_Bcast()



MPI_Bcast()

- Um padrão de comunicação que envolva todos os processos em um comunicador é chamada de comunicação coletiva.
- Uma difusão (**broadcast**) é uma comunicação coletiva na qual um único processo envia os mesmos dados para cada processo.
- A função MPI para difusão é:

```
int MPI_Bcast (void* mensagem, int cont, MPI_Datatype tipo_mpi, int raiz,  
MPI_Comm com)
```

MPI_Bcast()

- Ela simplesmente envia uma cópia dos dados de mensagem no processo **raiz** para cada processo no comunicador **com**.
- Para que a operação funcione corretamente, ela deve ser chamado por **todos** os processos no comunicador com os mesmos argumentos para **raiz** e **com**.
- Uma mensagem de difusão **não** pode ser recebida com **MPI_Recv()**, pois a operação é exclusivamente coletiva.
- Os parâmetros **cont** e **tipo_mpi** têm a mesma função que nas funções **MPI_Send()** e **MPI_Recv()**, definindo o tamanho e o tipo da mensagem.
- Contudo, ao contrário das funções ponto-a-ponto, o padrão MPI exige que os valores de **cont** e **tipo_mpi** sejam **iguais para todos os processos** dentro do mesmo comunicador em uma comunicação coletiva.

Exemplo

```
#include <stdio.h>
#include "mpi.h"
int main(int argc, char *argv[]) { /* mpi_bcast.c */
    int valor, meu_ranque, num_procs, raiz = 0;
    MPI_Comm comm=MPI_COMM_WORLD;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(comm, &meu_ranque);
    MPI_Comm_size(comm, &num_procs);
    /* Cada processo tem um valor inicial diferente */
    valor = meu_ranque;
    printf("0 processo com ranque %d tem inicialmente o valor: %d\n", meu_ranque,
valor);
    /* O valor a ser enviado é lido pelo processo raiz */
    if (meu_ranque == raiz) {
        printf("Entre um valor: \n");
        scanf("%d", &valor);
        MPI_Barrier(comm);
    }
}
```

Exemplo

```
else
    MPI_Barrier(comm);
    printf("Passei da barreira. Eu sou o %d de %d processos \n", meu_ranke, num
_procs);
    /* A rotina de difusão deve ser chamada por todos processos */
    /* Apenas o processo raiz envia, mas todos recebem o valor enviado*/
    MPI_Bcast(&valor, 1, MPI_INT, raiz, comm);
    /* O valor agora é o mesmo em todos os processos */
    printf("O processo com ranque %d recebeu o valor: %d\n", meu_ranke, valor);
    /* Termina a execução */
    MPI_Finalize();
    return 0;
}
```

Exemplo

```
$ mpirun -np 3 ./teste
```

O processo com ranque 0 tem inicialmente o valor: 0

Entre um valor:

O processo com ranque 1 tem inicialmente o valor: 1

O processo com ranque 2 tem inicialmente o valor: 2

33

Passei da barreira. Eu sou o 0 de 3 processos

O processo com ranque 0 recebeu o valor: 33

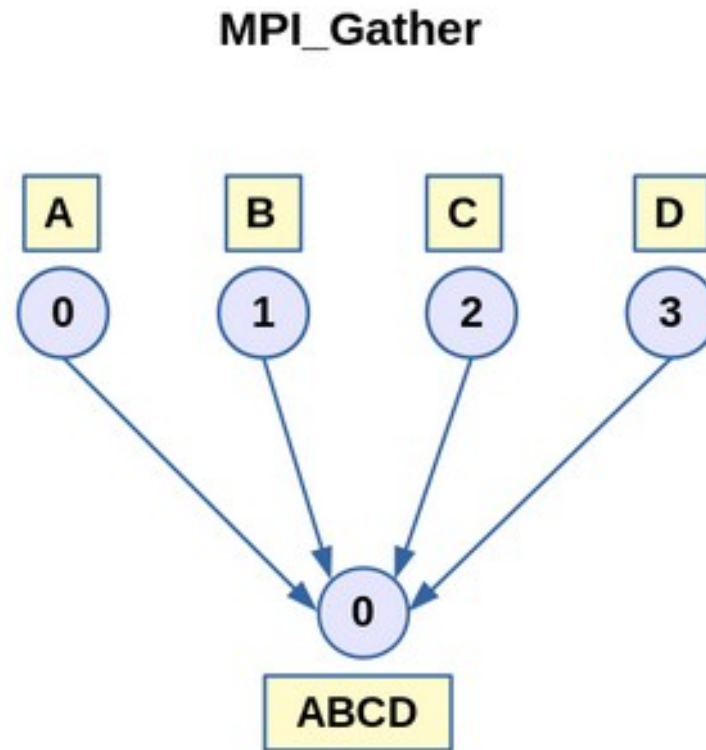
Passei da barreira. Eu sou o 1 de 3 processos

O processo com ranque 1 recebeu o valor: 33

Passei da barreira. Eu sou o 2 de 3 processos

O processo com ranque 2 recebeu o valor: 33

MPI_Gather()



MPI_Gather()

```
int MPI_Gather(void* vet_envia, int cont_envia, MPI_Datatype tipo_envia,  
void* vet_recebe, int cont_recebe, MPI_Datatype tipo_recebe, int raiz,  
MPI_comm com)
```

- Cada processo no comunicador **com** envia o conteúdo de **vet_envia** para o processo com ranque igual a **raiz**.
- O processo **raiz** concatena os dados que são recebidos em **vet_recebe** em uma ordem que é definida pelo **ranque** de cada processo.
- Os argumentos que terminam com **recebe** são significativos apenas no processo com ranque igual a **raiz**.
- O argumento **cont_recebe** indica o número de itens enviados por **cada processo**, não número total de itens recebidos pelo processo **raiz** e, normalmente, é igual a **cont_envia**.

Exemplo

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"
#define TAM_VET 10

int main(int argc, char *argv[]) { /* mpi_gather.c */
    int meu_ranque, num_procs, raiz = 2;
    int *vet_recebe, vet_envia[TAM_VET];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &meu_ranque);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
    /* O processo raiz aloca o espaço de memória para o vetor de recepção */
    if (meu_ranque == raiz)
        vet_recebe = (int *) malloc(num_procs*TAM_VET*sizeof(int));
    /* Cada processo atribui valor inicial ao vetor de envio */
    for (int i = 0; i < TAM_VET; i++)
        vet_envia[i] = meu_ranque;
    /* O vetor é recebido pelo processo raiz com as partes recebidas
       de todos processos, incluindo o processo raiz */
    MPI_Gather (vet_envia, TAM_VET, MPI_INT, vet_recebe, TAM_VET, MPI_INT, raiz, MPI_COMM_WORLD);
```

Exemplo

```
/* 0 processo raiz imprime o vetor recebido */
if (meu_ranque == raiz) {
    printf("Processo = %d, recebeu", meu_ranque);
    for (int i = 0; i < (TAM_VET*num_procs); i++) {
        printf(" %d", vet_recebe[i]);
    }
    printf("\n");
}
/* Termina a execução */
MPI_Finalize();
return(0);
```

Exemplo

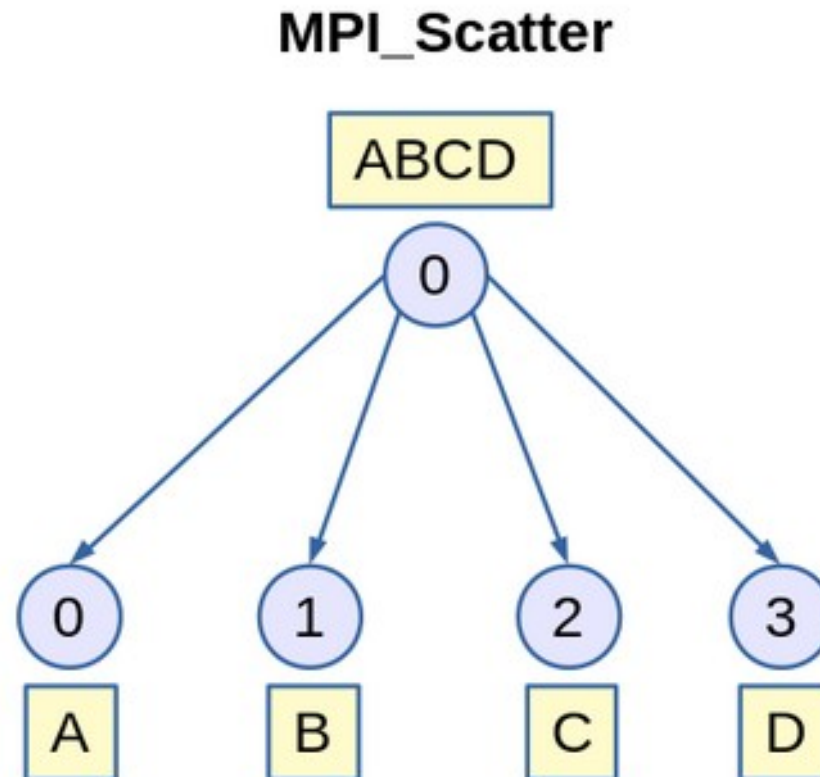
- A execução do programa com os parâmetros abaixo terá o seguinte resultado:

```
$ mpicc -o teste mpi_gather.c
```

```
$ mpirun -np 4 ./teste
```

```
Processo = 2, recebeu 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 3 3 3 3 3  
3 3 3 3 3
```

MPI_Scatter()



MPI_Scatter()

```
int MPI_Scatter(void* vet_envia, int cont_envia, MPI_Datatype tipo_envia, void*  
vet_recebe, int cont_recebe, MPI_Datatype tipo_recebe, int raiz, MPI_Comm com)
```

- O processo com o ranque igual a **raiz** distribui o conteúdo de **vet_envia** entre os **p** processos.
- O conteúdo de **vet_envia** é dividido em **p** segmentos, cada um deles consistindo de **cont_envia** itens.
- O primeiro segmento vai para o processo com ranque **0**, o segundo para o processo com ranque **1**, etc.
- Os argumentos que terminam com **envia** são significativos apenas no processo **raiz**, pois ele é o único que possui o conteúdo completo a ser distribuído.

Exemplo

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"
#define TAM_VET 10

int main(int argc, char *argv[]) { /* mpi_scatter.c */
    int i, meu_ranque, num_procs, raiz = 0;
    int *vet_envia, vet_recebe[TAM_VET];
    MPI_Comm com=MPI_COMM_WORLD;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(com, &meu_ranque);
    MPI_Comm_size(com, &num_procs);
    /* O processo raiz aloca o espaço de memória e inicia o vetor */
    if (meu_ranque == raiz) {
        vet_envia = (int*) malloc (num_procs*TAM_VET*sizeof(int));
        for (i = 0; i < (TAM_VET*num_procs); i++)
            vet_envia[i] = i;
    }
}
```

Exemplo

```
/* O vetor é distribuído em partes iguais entre os processos,
   incluindo o processo raiz */
MPI_Scatter(vet_envia, TAM_VET, MPI_INT, vet_recebe, TAM_VET, MPI_INT, raiz,
com);
/* Cada processo imprime a parte que recebeu */
printf("Processo = %d, recebeu", meu_ranque);
for (i = 0; i < TAM_VET; i++)
    printf(" %d", vet_recebe[i]);
printf("\n");
/* Termina a execução */
MPI_Finalize();
return(0);
}
```


Exemplo

```
$ mpicc -o teste mpi_scatter.c
```

```
$ mpirun -np 4 ./bin/mpi_scatter
```

```
Processo = 0, recebeu 0 1 2 3 4 5 6 7 8 9
```

```
Processo = 1, recebeu 10 11 12 13 14 15 16 17 18 19
```

```
Processo = 2, recebeu 20 21 22 23 24 25 26 27 28 29
```

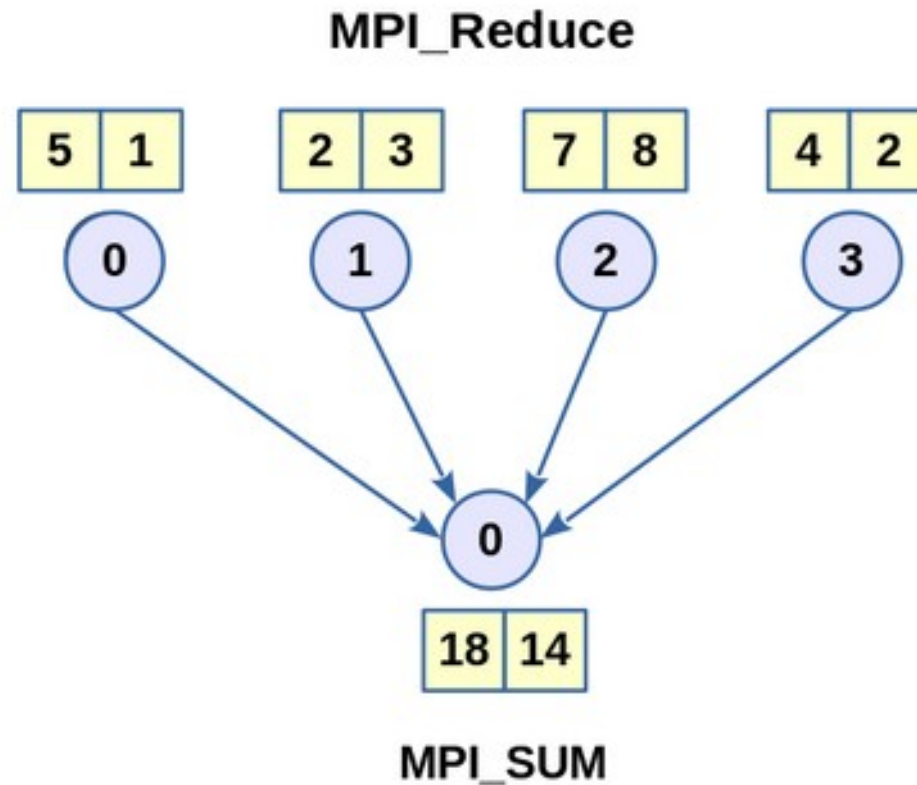
```
Processo = 3, recebeu 30 31 32 33 34 35 36 37 38 39
```



Escola Supercomputador Santos Dumont - 2026

Comunicação Coletiva

MPI_Reduce()



MPI_Reduce()

- Realiza, por exemplo, uma soma global, que é um exemplo de uma classe geral de operações de comunicação coletivas chamada operações de **redução**.
- Em uma operação global de redução, todos os processos em um comunicador contribuem com dados que são combinados em operações binárias.
- Operações binárias típicas são a adição, máximo, mínimo, e lógico, etc.
- É possível definir operações adicionais além das mostradas para a função **MPI_Reduce()**.

MPI_Reduce()

```
int MPI_Reduce(void* operando, void* resultado, int cont, MPI_Datatype  
tipo_mpi, MPI_Op oper, int raiz, MPI_Comm com)
```

- A operação `MPI_Reduce()` combina os operandos armazenados em `*operando` usando a operação `oper` e armazena o resultado em `*resultado` no processo `raiz`.
- Tanto `operando` como `resultado` referem-se a `cont` posições de memória com o tipo `tipo_mpi`.
- `MPI_Reduce()` deve ser chamada por todos os processos no comunicador `com` e os valores de `cont`, `tipo_mpi` e `oper` devem ser os mesmos em cada processo.

Valores do argumento oper

MPI_MAX		Máximo
MPI_MIN	Mínimo	
MPI_SUM		Soma
MPI_PROD	Produto	
MPI_LAND		“E” lógico
MPI_BAND	“E” bit a bit	
MPI_LOR	“Ou” lógico	
MPI_BOR		“Ou” bit a bit
MPI_LXOR		“Ou Exclusivo” lógico
MPI_BXOR		“Ou Exclusivo” bit a bit
MPI_MAXLOC	Máximo e Posição do Máximo	
MPI_MINLOC	Mínimo e Posição do Mínimo	

Exemplo

```
#include <stdio.h>
#include <math.h>
#include "mpi.h"
#define TAM 5

int main(int argc, char *argv[]) { /* mpi_reduce.c */
    int meu_ranque, num_procs, i, raiz = 0;
    float vet_envia [TAM] ; /* Vetor a ser enviado */
    float vet_recebe [TAM]; /* Vetor a ser recebido */

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &meu_ranque);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
    /* Preenche o vetor com valores que dependem do ranque */
    for (i = 0; i < TAM; i++)
        vet_envia[i] = (float) (meu_ranque*TAM+i);
    /* Faz a redução, encontrando o valor máximo do vetor */
    MPI_Reduce(vet_envia, vet_recebe, TAM, MPI_FLOAT, MPI_MAX, raiz, MPI_COMM_WORLD);
    /* O processo raiz imprime o resultado da operação de redução */
    if (meu_ranque == raiz) {
        for (i = 0; i < TAM; i++)
            printf("vet_recebe[%d] = %3.1f ", i, vet_recebe[i]);
        printf("\n\n");
    }
    MPI_Finalize();
    return(0);
}
```

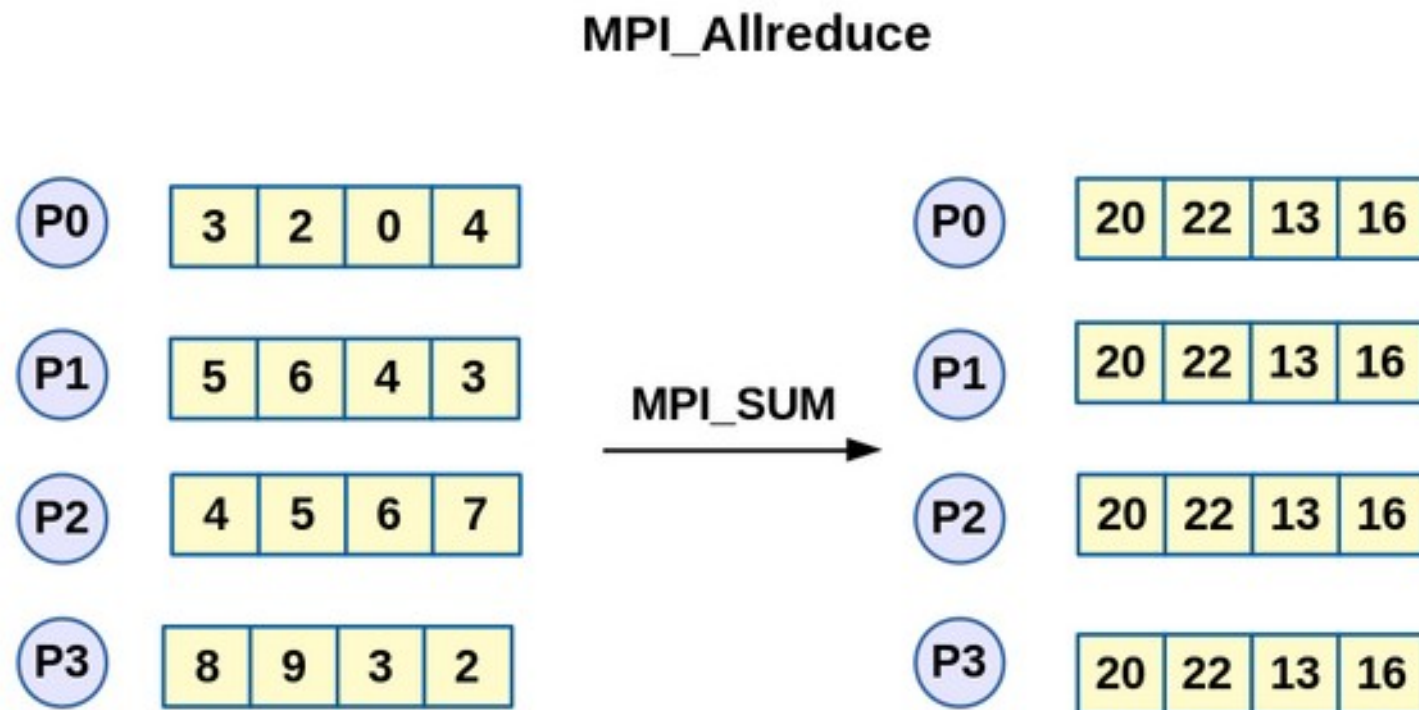
Exemplo MPI_Reduce()

```
$ mpicc -o teste mpi_reduce.c
```

```
$ mpirun -np 4 ./bin/mpi_reduce
```

```
vet_recebe[0] = 15.0 vet_recebe[1] = 16.0 vet_recebe[2] = 17.0 vet_recebe[3] = 18.0 vet_recebe[4]  
= 19.0
```


MPI_Allreduce()



MPI_Allreduce()

int MPI_Allreduce (void* vet_envia, void* vet_recebe, int cont, MPI_Datatype tipo_mpi, MPI_Op oper, MPI_Comm com)

- **MPI_Allreduce()** tem um resultado similar à operação **MPI_Reduce()**, com a diferença que o resultado da operação de redução **oper** é armazenado no vetor **vet_recebe** de cada processo envolvido na chamada e não apenas no processo **raiz**.
- Aliás, como todos recebem o resultado da redução, não há necessidade do parâmetro **raiz** no protótipo da função.

Cálculo de Pi

- O valor de π pode ser obtido pela integração numérica :

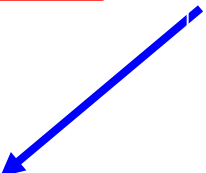
$$\int_0^1 \frac{4}{1+x^2}$$

- Que pode ser calculada em paralelo, dividindo-se o intervalo de integração entre vários processos, de modo similar ao método do trapézio.

Cálculo de Pi

```
#include<stdio.h>
#include <math.h>
#include "mpi.h"
int main (int argc, char *argv[]){
int meu_ranque, num_procs, n=10000000;
double mypi, pi, h, x, sum = 0.0;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
    MPI_Comm_rank(MPI_COMM_WORLD, &meu_ranque);
    h=1.0/(double) n;
    for (int i = meu_ranque +1; i <= n; i += num_procs){
        x = h * ((double) i - 0.5);
        sum += (4.0/(1.0 + x*x));
    }
    mypi = h* sum;
    MPI_Allreduce(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
    printf ("valor aproximado de pi: %.16f \n", pi);
    MPI_Finalize( );
}
```

Não tem
raiz



Exemplo

```
$ mpicc -o teste mpi_calcpio.c
```

```
$ mpirun -np 4 ./teste
```

```
valor aproximado de pi: 3.1415926535896865
```

```
valor aproximado de pi: 3.1415926535896865
```

```
valor aproximado de pi: 3.1415926535896865
```

```
valor aproximado de pi: 3.1415926535896865
```



Escola Supercomputador Santos Dumont - 2026

Programação com MPI

Escola Supercomputador Santos Dumont 2026

Ementa

- Estudo de caso: método do trapézio
- Estudo de caso: multiplicação de matriz
- Estudo de caso: número primos

Escola Supercomputador Santos Dumont 2026

Bibliografia



<https://www.casadocodigo.com.br/products/livro-programacao-paralela>

Escola de Computação Santos Dumont - 2026

Google Colab

- Os exemplos, notebooks e slides do minicurso estão disponíveis no seguinte endereço:
- <https://github.com/Programacao-Paralela-e-Distribuida/MPI/>

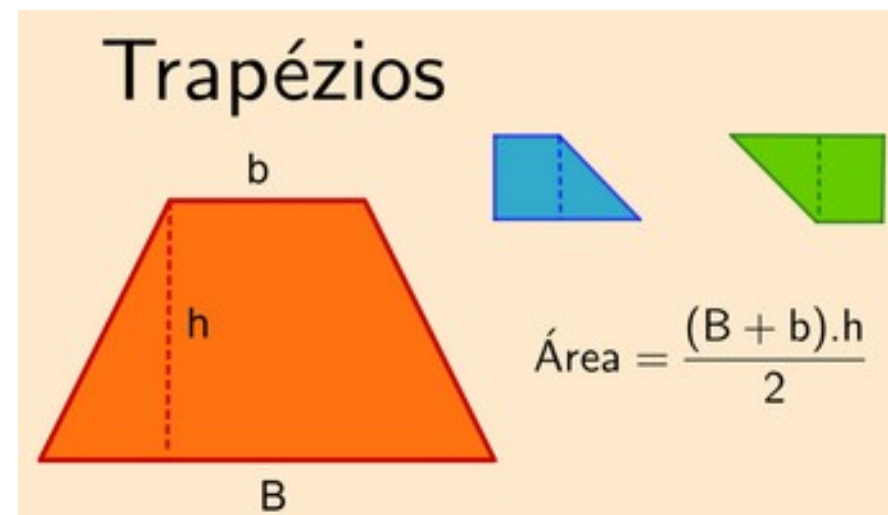
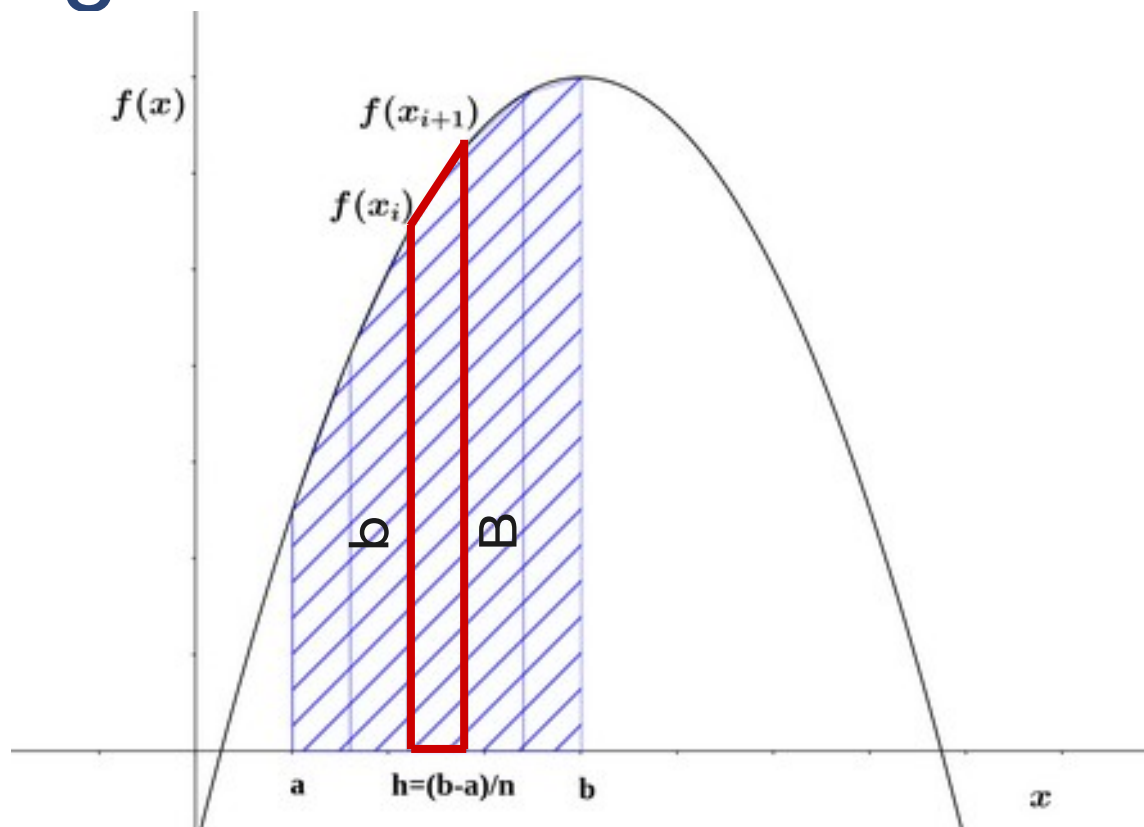


Método do Trapézio

Integral Definida - Método do Trapézio

- O valor de uma integral definida pode ser aproximado por vários métodos numéricos.
- Um dos métodos mais utilizados é o método do trapézio, onde o valor da integral de uma função, que é definido como a área sob a curva da função até o eixo x , é aproximado pela soma da área de diversos trapézios.
- Vejamos uma ilustração no slide a seguir.

Integral Definida - Método do Trapézio



Integral Definida - Método do Trapézio

- Vamos lembrar que o método do trapézio estima o valor de $f(x)$ dividindo o intervalo $[a; b]$ em n segmentos iguais e calculando a seguinte soma:

$$h * \left[\frac{f(x_0)}{2} + \frac{f(x_n)}{2} + \sum_{i=1}^{n-1} f(x_i) \right] \quad (3.1)$$

$$h = \frac{(b-a)}{n} \text{ e } x_i = a + i * h, i = 1, \dots, (n - 1)$$

- Colocando $f(x)$ em uma rotina, podemos escrever um programa para calcular uma integral utilizando o método do trapézio.

Versão Sequencial

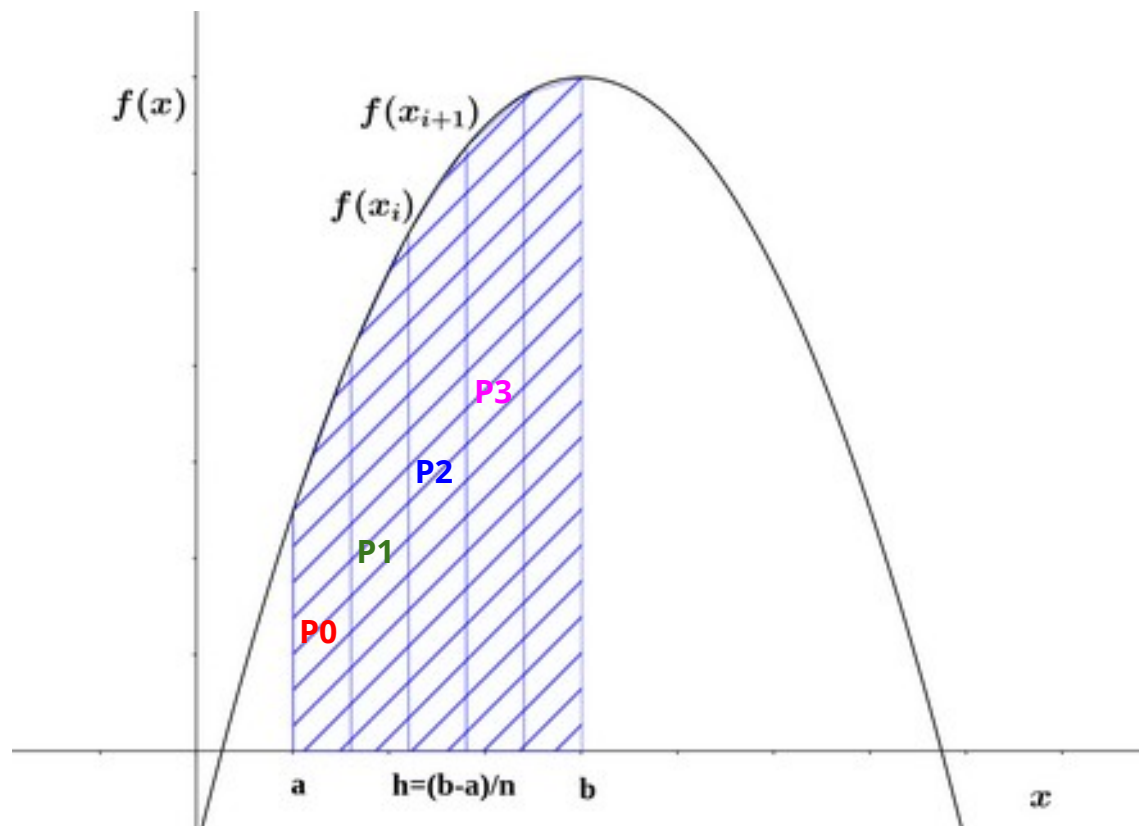
```
#include <stdio.h>
#include <math.h>
float f(float x) { /* Calcula f(x). */
    float return_val;
    return_val=exp(x);
    return return_val;
}
int main(int argc, char *argv[]) { /* trapezio_seq.c */
    float integral; /* integral armazena resultado final */
    float a, b; /* a, b - limite esquerdo e direito da função */
    int i,n; /* n - número de trapezóides */
    float x, h; /* h - largura da base do trapezóide */

    printf("Entre a, b, e n \n");
    scanf("%f %f %d", &a, &b, &n);
    h = (b-a)/n;
    integral = (f(a) + f(b))/2.0;
    x = a;
    for (i = 1; i < n; i++) {
        x += h;
        integral += f(x);
    }
    integral *= h;
    printf("Com n = %d trapezóides, a estimativa \n", n);
    printf("da integral de %f até %f = %f\n", a, b, integral);
    return(0);
}
```

Método do Trapézio

- Este método divide o espaço de integração em **N** intervalos de tamanho **h**, onde **h** = **(b - a) / N**.
- Uma maneira simples de distribuir o trabalho entre os diversos processadores seria atribuir **N/P** intervalos em bloco para cada processo.
- Contudo, esse método tem dificuldade quando **N** não é múltiplo de **P**, e pode apresentar algum desbalanceamento de carga.
- Assim, uma melhor forma de distribuição é atribuir os intervalos alternadamente a cada processo, como ilustrado na figura seguinte.

Integral Definida - Método do Trapézio



Método do Trapézio

- Essa forma de distribuição pode ser realizada facilmente atribuindo o primeiro intervalo para cada processo **p** com a fórmula:

$$x_p = a + h * (\text{ranque} + 1)$$

- E saltando **num_proc** intervalos **h** até que o valor de **x** seja maior do que **b**.
- Assim, mesmo que **N** não seja múltiplo de **P**, as iterações restantes são distribuídas o mais igualmente possível entre os processos, melhorando assim o balanceamento de carga.

Escola Supercomputador Santos Dumont 2026

Versão Paralela

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &meu_ranque);
MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
/* h é o mesmo para todos os processos */
h = (b - a)/n;
/* O processo 0 calcula o valor de f(x)/2 em a e b */
if (meu_ranque == 0) {
    tempo_inicial = MPI_Wtime();
    integral = (f(a) + f(b))/2.0;
}
/* Cada processo calcula a integral sobre n/num_procs trapézios */
for (x = a+h*(meu_ranque+1); x < b ; x += num_procs*h) {
    integral += f(x);
}
integral = integral*h;
```

Versão Paralela

```
/* O processo 0 soma as integrais parciais recebidas */
if (meu_ranque == 0) {
    total = integral;
    for (origem = 1; origem < num_procs; origem++) {
        MPI_Recv(&integral, 1, MPI_DOUBLE, origem, etiq,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        total += integral;
    }
} else {
    /* As integrais parciais são enviadas para o processo 0 */
    MPI_Send(&integral, 1, MPI_DOUBLE, destino, etiq,
MPI_COMM_WORLD);
}
/* Imprime o resultado */
if (meu_ranque == 0) {
    tempo_final = MPI_Wtime();
    printf("Foram gastos %3.1f segundos\n", tempo_final-
tempo_inicial);
    printf("Com n = %ld trapezoides, a estimativa\n", n);
    printf("da integral de %lf até %lf = %lf \n", a, b, total);
}
MPI_Finalize();
return(0);
```

Otimizando o programa

- Lembre-se também que o MPI oferece uma operação coletiva de redução, que pode ser utilizada para simplificar o código e otimizar a execução em paralelo.
- Assim, podemos reescrever as últimas linhas do programa substituindo o laço de envio dos resultados parciais pela rotina de redução, que é uma forma muito mais eficiente para o cálculo do valor final da integral.

Escola Supercomputador Santos Dumont 2026

Versão Paralela Otimizada

```
/* Soma as integrais calculadas por cada processo */  
MPI_Reduce(&integral, &total, 1, MPI_FLOAT, MPI_SUM, 0, MPI_COMM_WORLD)  
;  
/* Imprime o resultado */  
if (meu_ranque == 0) {  
    tempo_final = MPI_Wtime();  
    printf("Foram gastos %3.1f segundos\n", tempo_final-  
tempo_inicial);  
    printf("Com n = %ld trapezoides, a estimativa\n", n);  
    printf("da integral de %lf até %lf = %lf \n", a, b, total);  
}  
MPI_Finalize();  
return(0);  
}
```

Método do trapézio

```
$ mpicc -o teste mpi_trapezio.c
```

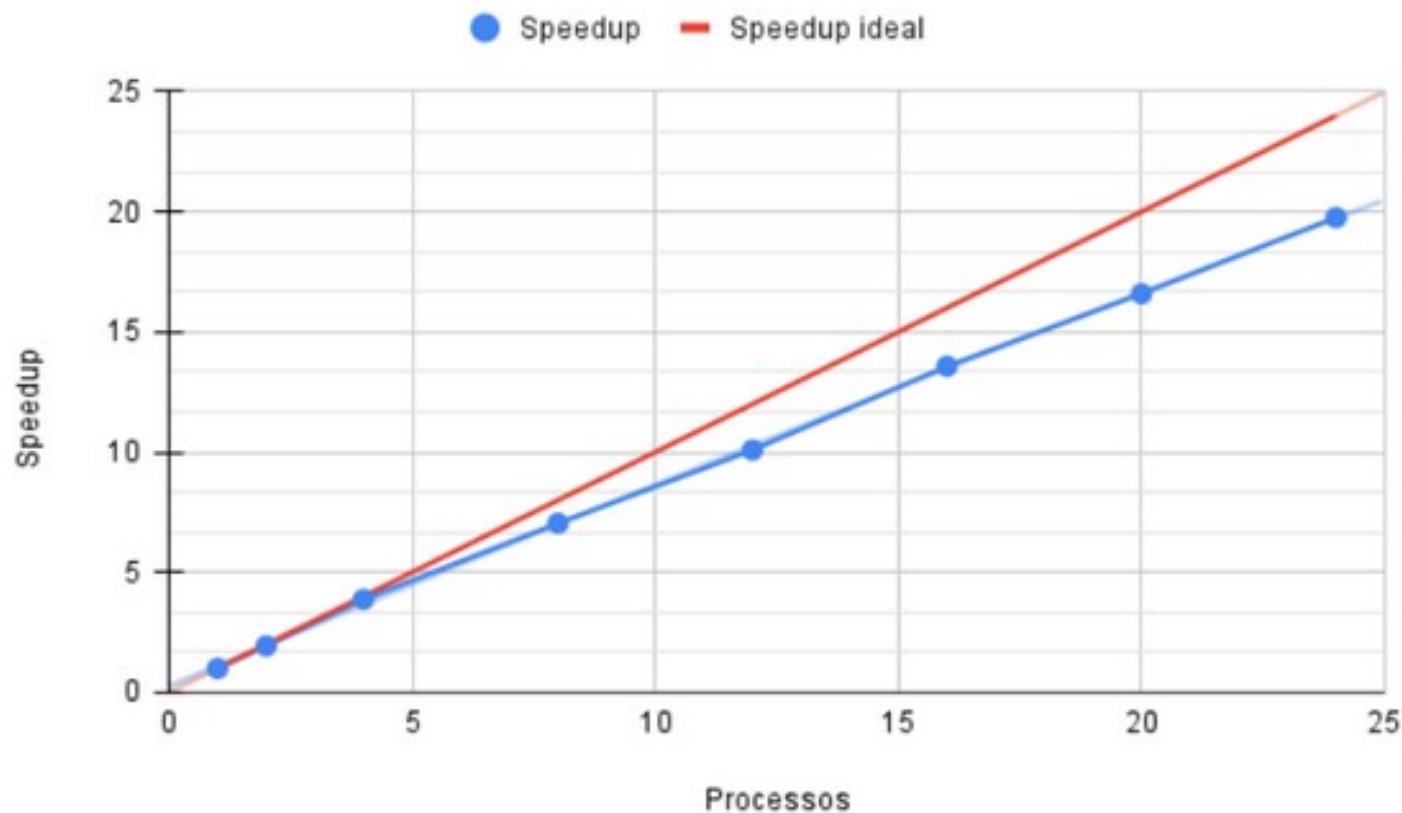
```
$ mpirun -np 4 ./bin/mpi_trapezio
```

Foram gastos 1.45700 segundos

Com $n = 500000000$ trapezoides, a estimativa da integral de 0.000000 até 1.000000 = 1.718282

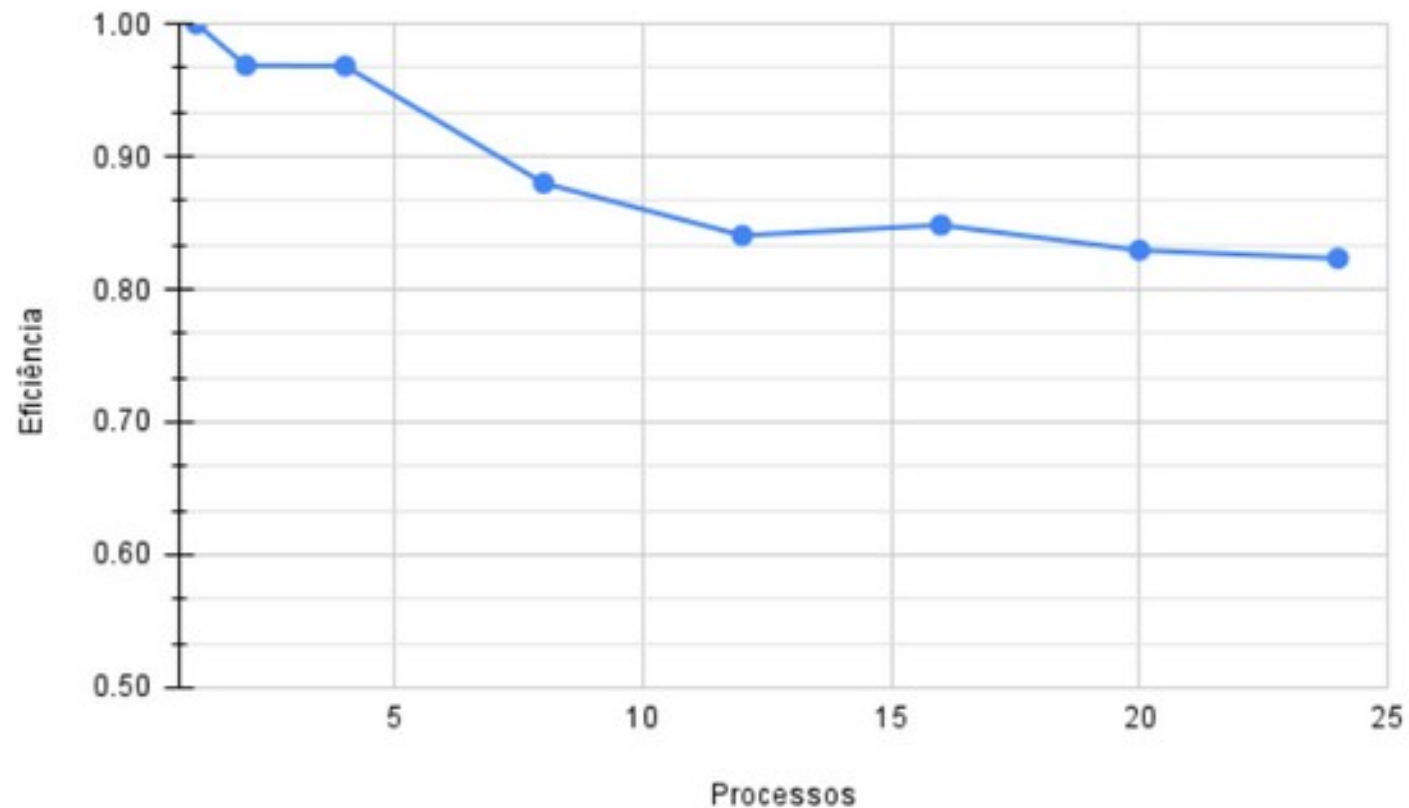
Escola Supercomputador Santos Dumont 2026

Speedup



Escola Supercomputador Santos Dumont 2026

Eficiência





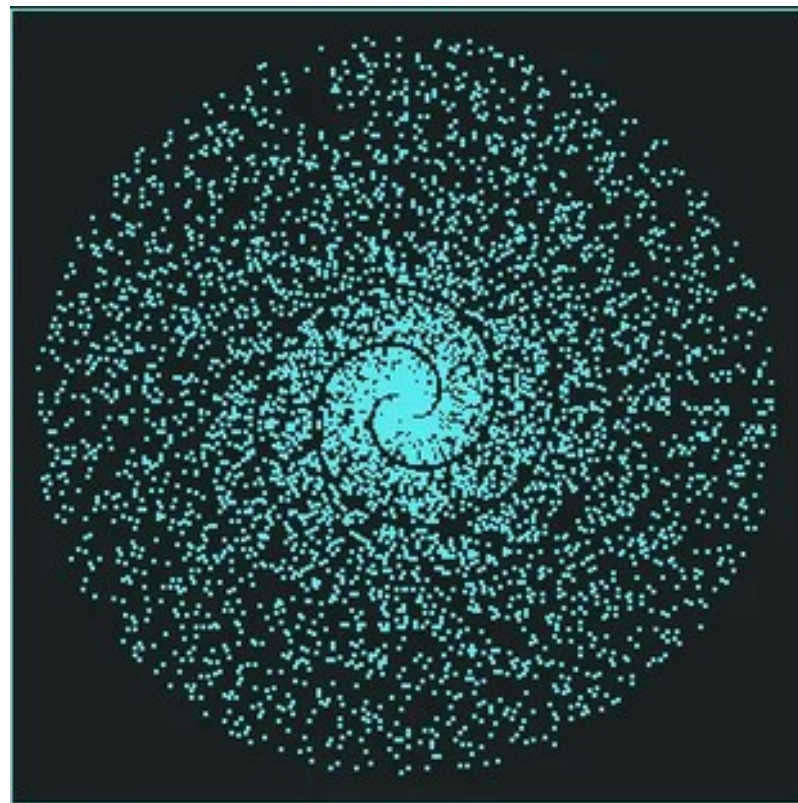
Escola Supercomputador Santos Dumont - 2026

Números Primos

Números primos

- O programa em questão serve para determinar a quantidade de números primos entre 0 e um determinado valor inteiro N .
- Embora possa parecer um programa trivial a princípio, ele tem algumas particularidades que o tornam um problema interessante.
- Na matemática, o Teorema do Número Primo (TNP) descreve a distribuição assintótica dos números primos entre os inteiros positivos.
- Ele formaliza a ideia intuitiva de que os números primos tornam-se menos comuns à medida que N aumenta, quantificando precisamente a taxa em que isso ocorre.

Distribuição de primos - coordenadas polares



Números primos

- O que isso implica na programação paralela?
- Bom, a nossa primeira tentativa de paralelização seria dividir o total de N números igualmente entre os P processadores disponíveis, ou seja, N/P valores para cada uma das *threads*.
- No entanto, há uma implicação importante: a distribuição dos números primos não é uniforme entre os inteiros.
- Isso torna essa abordagem de divisão direta ineficaz, pois as *threads* que receberem intervalos com uma maior concentração de números primos terão uma carga de trabalho significativamente maior.
- Isso pode levar a um desequilíbrio no desempenho, com algumas *threads* concluindo suas tarefas mais rapidamente, enquanto outras permanecem ocupadas, resultando em subutilização dos recursos disponíveis.

Números primos

- Mas antes de avançarmos nessas questões, vamos ver como se comporta o algoritmo sequencial de busca de primos.
- Essa solução descarta todos os números pares de início, pois obviamente eles não são primos.
- Em seguida, é verificado se N é divisível por algum número ímpar entre 0 e a raiz quadrada de N . Por que isso?
- Se um número composto N pode ser fatorado como $N = a * b$, então pelo menos um dos fatores (a ou b) deve ser menor ou igual à raiz quadrada de N .
- Se ambos a e b forem maiores que a raiz quadrada de N , então ao multiplicá-los, o resultado seria maior que N . Perfeito?

Números primos Sequencial

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int primo (long int n) {
    for (long int i = 3; i < (int)(sqrt(n) + 1); i+=2)
        if (n%i == 0) return 0;
    return 1;
}

int main(int argc, char *argv[]) { /* primos_seq.c */
    int total = 0;
    long int i, n=10000;
    if (argc > 1) n = strtol(argv[1], (char **) NULL, 10);
    for (i = 3; i <= n; i += 2)
        if (primo(i) == 1) total++;
    total += 1; /* Acrescenta o dois, que também é primo */
    printf("Quant. de primos entre 1 e n: %d \n", total);
    return(0);
}
```

Números primos

- Uma estratégia mais simples de paralelização seria distribuir as tarefas entre os processos em lote, passando uma faixa contínua de valores para cada processo verificar a quantidade de números primos no intervalo recebido.
- Mas como já vimos, essa não é uma boa abordagem, pois a distribuição de números primos não é uniforme ao longo do espaço de números inteiros.
- Um alternativa possível é cada processo verificar alternadamente se cada número ímpar é primo, o que na prática significa que cada processo comece verificando um número ímpar diferente e saltando **num_proc** números ímpares entre cada número ímpar verificado.

Números primos (paralelo)

```
início = 3 + meu_rank*2;
salto = num_procs*2;
for (i = início; i <= n; i += salto) {
    if(primo(i) == 1) cont++;
}
MPI_Reduce(&cont, &total, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
if (meu_rank == 0) {
    total += 1; /* Acrescenta o dois, que também é primo */
    printf("Quant. de primos entre 1 e n: %d \n", total);
}
MPI_Finalize();
return(0);
```

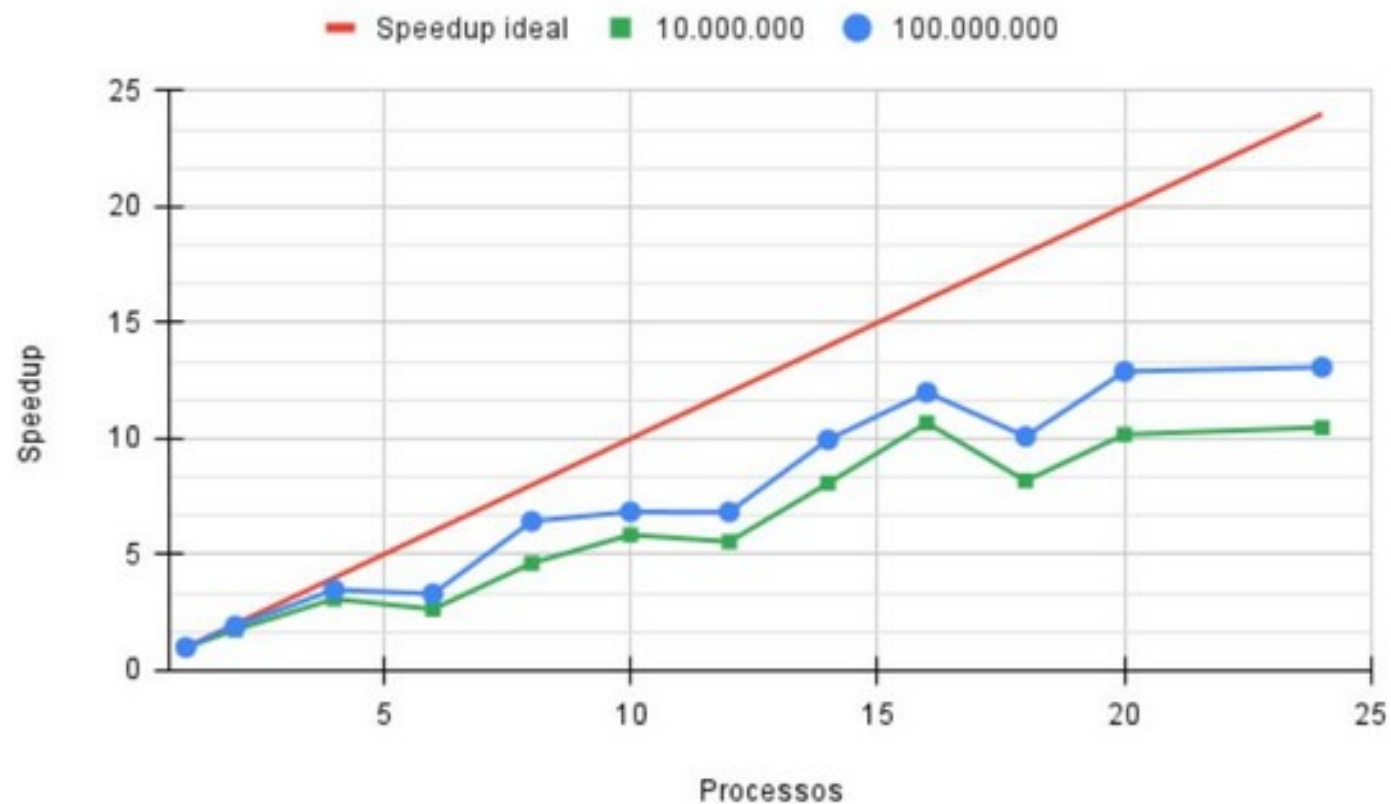

Escola Supercomputador Santos Dumont 2026

Números primos (paralelo)

```
$ mpicc -o teste mpi_primos.c -lm  
$ mpirun -np 4 ./teste  
  Quant. de primos entre 1 e n: 664579  
  Tempo de execucao: 3.418
```

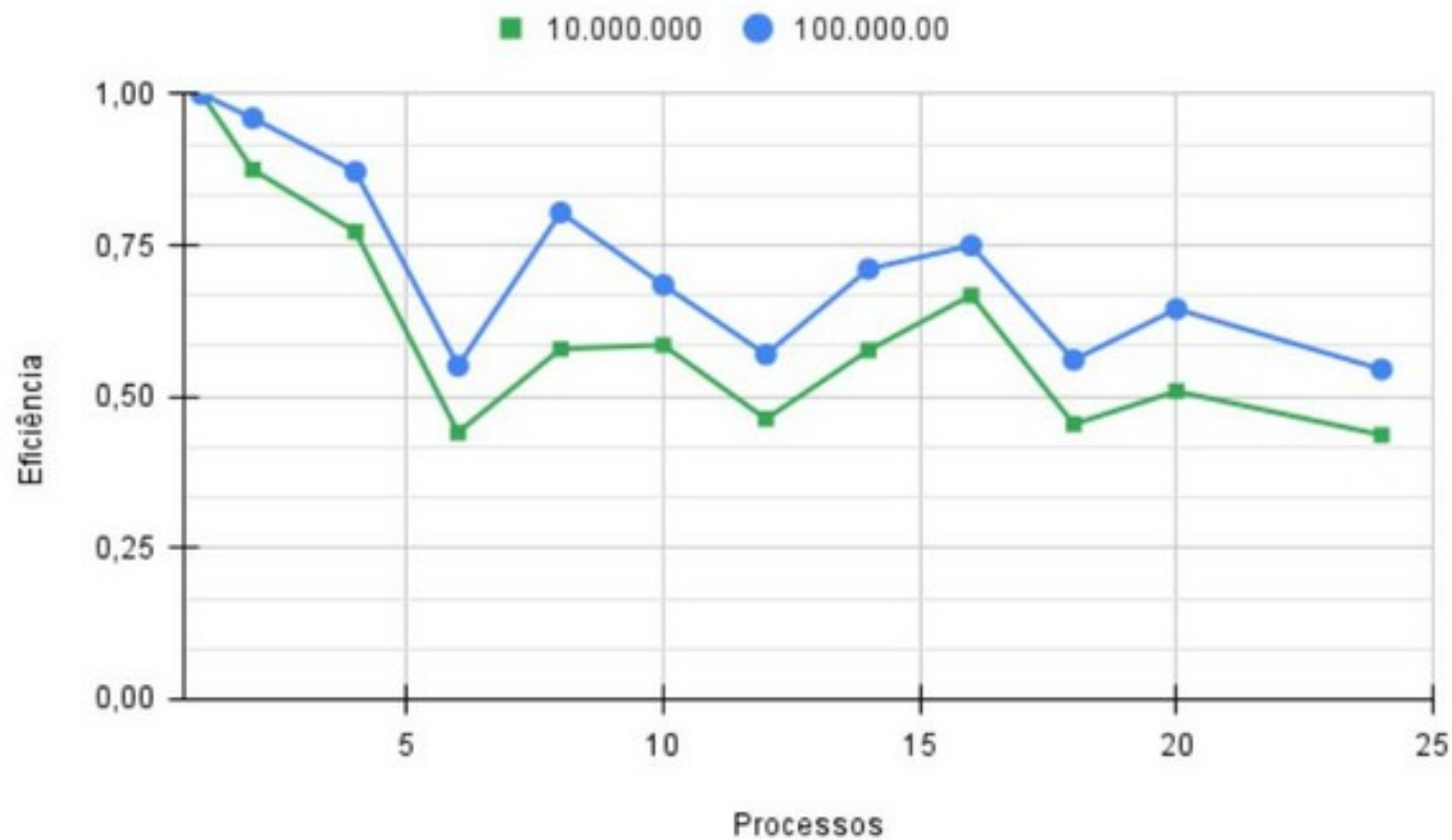
Escola Supercomputador Santos Dumont 2026

Speedup



Escola Supercomputador Santos Dumont 2026

Eficiência



Números primos (saco de tarefas)

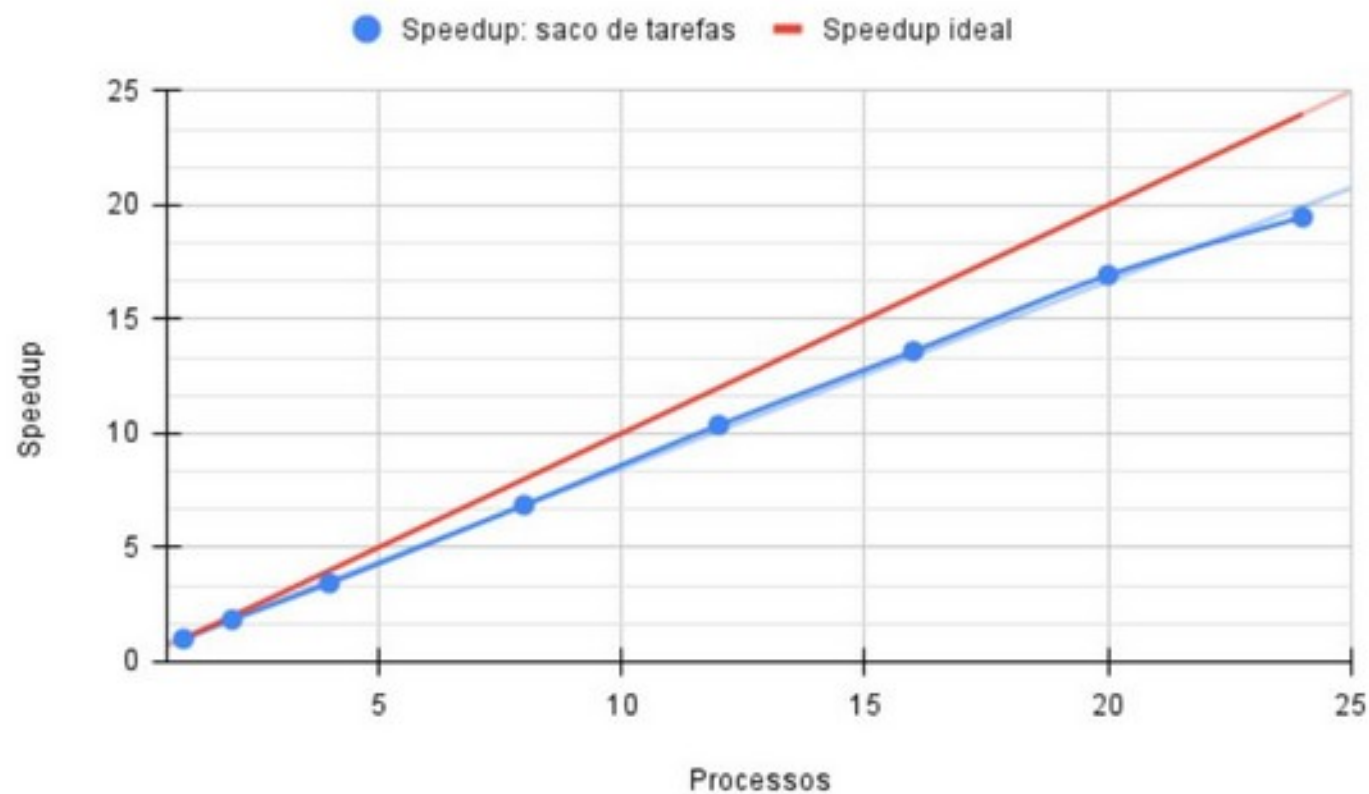
- Apesar dos cuidados que tomamos para a distribuição dos dados, há uma irregularidade muito grande no gráfico para valores do número de processadores iguais a 3, 5, 6, 7, 9, 10, 11, 12, 13 e 15.
- Curiosamente, nesses casos, existem processos que identificam apenas um ou nenhum número primo, terminando muito antes dos demais processos, que estão sobrecarregados calculando todos os demais números primos.
- Para vencer essa dificuldade é necessário empregar um outro método de paralelização chamado de “saco de tarefas”. Nesse método um processo (mestre) fica responsável por enviar as tarefas os demais processos (trabalhadores).

Números primos (paralelo)

- Assim que uma tarefa é terminada e o resultado enviado para o mestre, uma outra tarefa será alocada para o trabalhador e assim sucessivamente até que não haja mais tarefas para serem executadas.
- Maiores detalhes sobre esse tipo de implementação podem ser encontradas no livro texto.
- Apresentamos a seguir os gráficos com os resultados de speedup e eficiência deste método, que mostrou-se muito melhor.
- Isso mostra que a programação paralela não possui regras fixas, sendo que as soluções devem ser encontradas caso a caso.

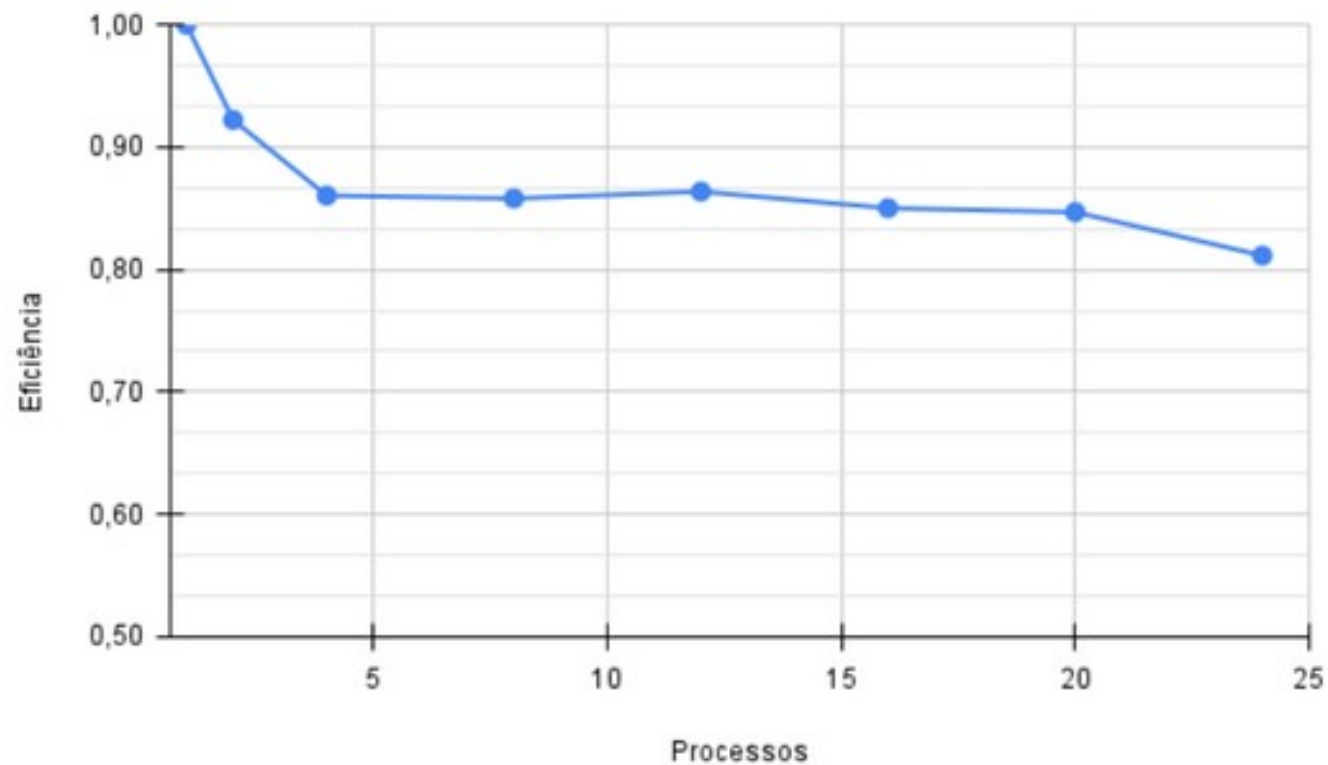
Escola Supercomputador Santos Dumont 2026

Speedup



Escola Supercomputador Santos Dumont 2026

Eficiência





Escola Supercomputador Santos Dumont - 2026

Multiplicação Matriz Vetor

Multiplicação Matriz-Vetor

- Um outro estudo de caso que faremos é a multiplicação de uma matriz por um vetor, que tem como resultado um vetor.

$$A_{m,n} = \begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n-1} \\ a_{2,0} & a_{2,1} & \cdots & a_{2,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix} \quad b_n = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_{n-1} \end{bmatrix}$$
$$c_m = Ab = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_{m-1} \end{bmatrix}$$

$$c_i = a_{i,0}.b_0 + a_{i,1}.b_1 + a_{i,2}.b_2 + \cdots + a_{i,n-1}.b_{n-1}$$

Multiplicação Matriz-Vetor

- Ao lado apresentamos um exemplo da operação.
- Note que cada elemento do vetor resultado é o produto escalar de uma linha da matriz com o vetor de entrada.

$$A \times b = c$$

2	1	3	4	0
5	-1	2	-2	4
0	3	4	1	2
2	3	1	-3	0

 $\times$

3
1
4
0
3

 $=$

19
34
25
13

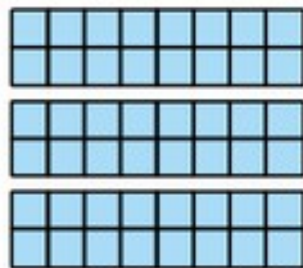
Código Sequencial

```
#include <stdio.h>
#include <stdlib.h>
void mxv(int m, int n, double* A, double* b, double* c)
{
    int i, j;
    for (i = 0; i < m; i++) {
        c[i] = 0.0;
        for (j = 0; j < n; j++)
            c[i] += A[i*n + j]*b[j];
    }
}

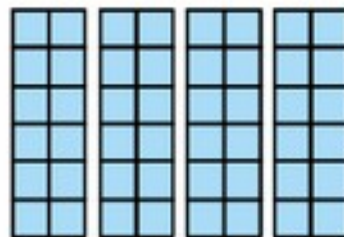
int main(int argc, char *argv[]) { /* mxv_seq.c */
    double *A, *b, *c;
    int i, j, m, n;
    printf("Por favor entre com m e n: ");
    scanf("%d %d", &m, &n);
    printf("\n");
    A=(double *)malloc(m*n*sizeof(double));
    b=(double *)malloc(n*sizeof(double));
    c=(double *)malloc(m*sizeof(double));
    printf("Atribuindo valor inicial à matriz A e ao vetor b\n");
    for (j = 0; j < n; j++)
        b[j] = 2.0;
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
            A[i*n + j] = i;
    printf("Mutiplicando a matriz A com o vetor b\n");
    (void) mxv(m, n, A, b, c);
    free(A);
    free(b);
    free(c);
    return(0);
}
```

Particionamento dos dados

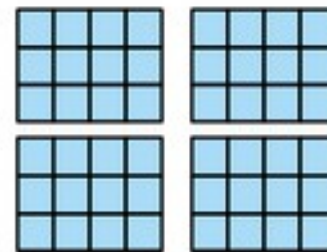
- Há várias maneiras de decompor o problema e distribuir os dados entre os diversos processos.



Decomposição em blocos no sentido das linhas



Decomposição em blocos no sentido das colunas



Decomposição em blocos $j \times k$

Particionamento dos dados

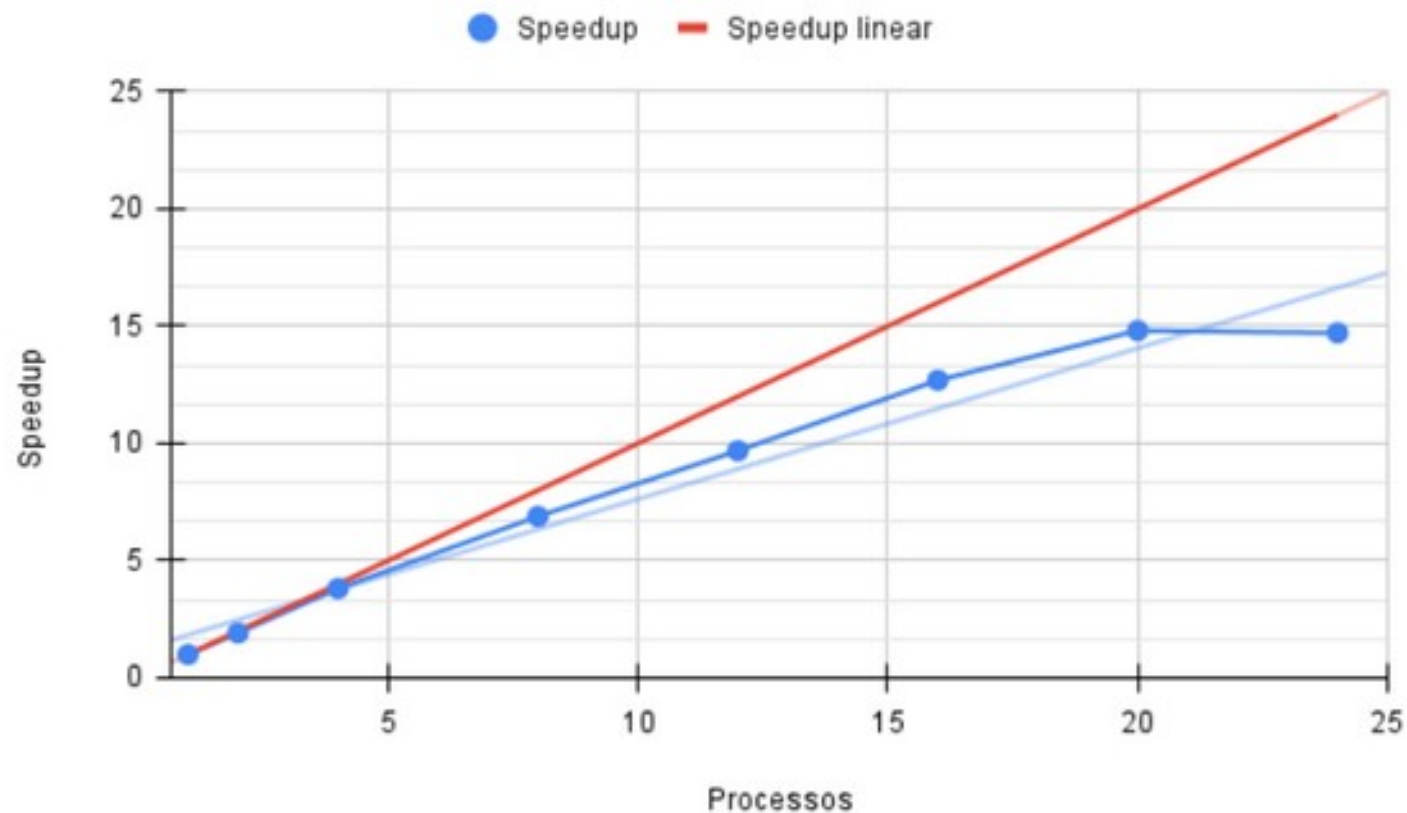
- Cada uma dessas formas de decomposição tem as suas vantagens e desvantagens, além de complexidades distintas.
- Por simplicidade, vamos assumir que utilizaremos a primeira alternativa de distribuição de dados, com cada processo possuindo um bloco de linhas da matriz **A** e os vetores **b** e **c** replicados em cada processo.
- Uma análise simples de complexidade, supondo-se $m = n$, indica uma complexidade computacional sequencial de $O(n^2)$. Quando p processos são utilizados, a complexidade computacional por processo, sem custos de comunicação, igual a $O(n^2 / p)$.

Código Paralelo

```
início = MPI_Wtime();  
    /* Difunde o vetor b para todos os processos */  
    MPI_Bcast(&b[0], n, MPI_DOUBLE, raiz, MPI_COMM_WORLD);  
    /* Distribui as linhas da matriz A entre todos os processos */  
    MPI_Scatter(A, n, MPI_DOUBLE, Aloc, n, MPI_DOUBLE, raiz, MPI_COMM_WORLD);  
    /* Cada processo faz o cálculo parcial de 'c' */  
    (void) mxv(n, Aloc, b, cloc);  
    /* O processo raiz coleta o vetor 'c' dos demais processos */  
    MPI_Gather(cloc, 1, MPI_DOUBLE, c, 1, MPI_DOUBLE, raiz, MPI_COMM_WORLD);  
fim = MPI_Wtime();
```

Escola Supercomputador Santos Dumont 2026

Speedup



Escola Supercomputador Santos Dumont 2026

Eficiência

